Final Project:
Rescue USB Stick

Student full name: **Yasaman Ghragozlou**

Student Number: **W0522396**

Program: **IT Programming – Group A**

Instructor: **Ryan Smith**

Course: **OSYS1000/1501/Operating Systems**

Institution: **Nova Scotia Community College, IT Campus**

Contents

Introduction 3

OS installation & configuration 4

System updates 10

App installation & configuration 13

Scripts 19

Creating a bootable ISO 40

Project functionality demo 45

Issues & solutions 58

Comprehensive Glossary of Commands 59

References 63

Introduction

In this assignment, the goal was to create a Rescue USB that would boot from the USB and immediately show a menu from which the user can choose to use different sets of tools for different rescue purposes.

I installed Debian 13 minimal with no GUI, as I researched and found out that this distro and version has a high compatibility with different hardware, is stable and light weight, and has access to special tools for a live build (live-build), but due to difficulties in primary configurations and listing packages, I moved on from creating the final ISO directly from this system. Instead, I downloaded and extracted Debian minimal (command line) live ISO, added my files and configurations and packages to it, and repackaged it to be a bootable live ISO, as this is the ideal format for a Rescue USB.

I developed scripts for each option on the menu to create an automated system that helps the user diagnose disk failures, repair partitions and file systems, create image backup or simple recovery of their files/directories, open text editors, check for malware in their files, check network connectivity, and hardware information.

Then I used different tools and tried different methods in order to create a bootable ISO file that can be run live.

Although the automated system turned out functional for the most part, there are some areas of improvement in the creation of the ISO file and burning it to a USB flash drive.

I created multiple snapshots and copies of the scripts, Gold Copy of my VM, and the final ISO file to ensure strong backup of data.

This document contains steps to creation of this system, scripts that I wrote, and demo of the final ISO being booted on a VM as a live system with no install.

OS installation & configuration

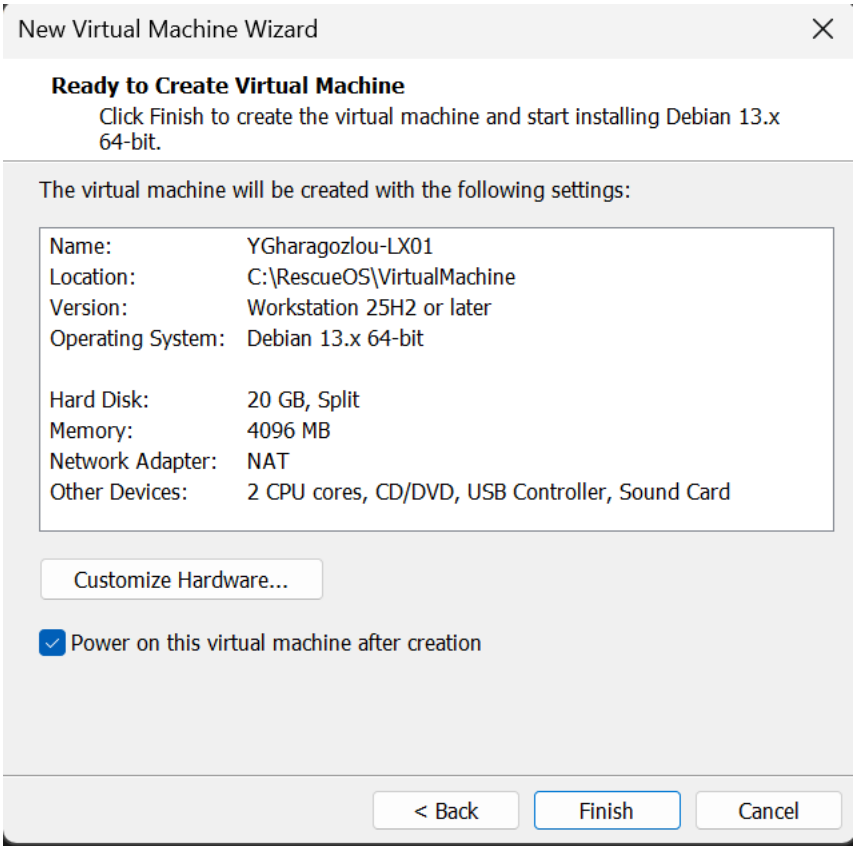


Figure 1 - created a new VM with these specifications

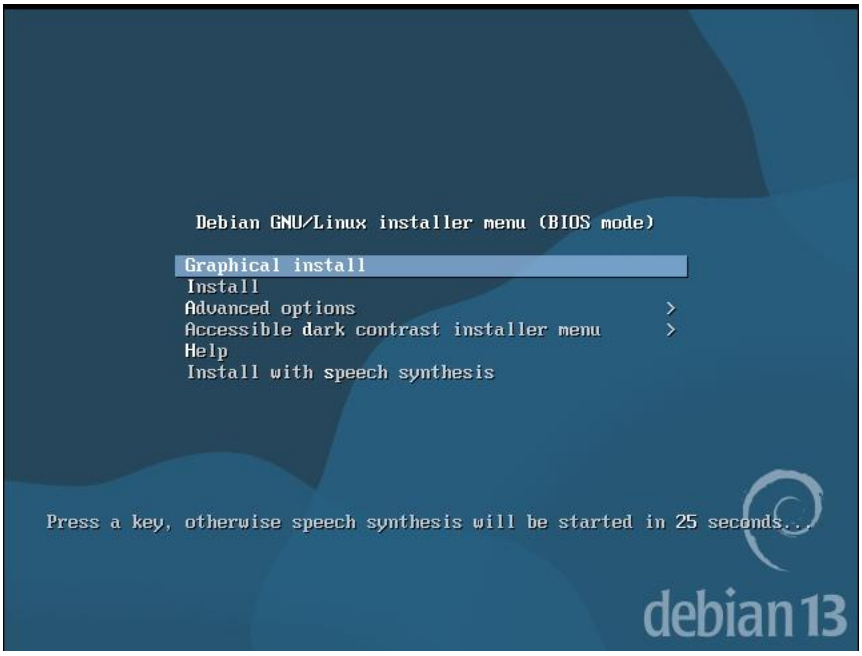


Figure 2 - installation of Debian 13 minimal

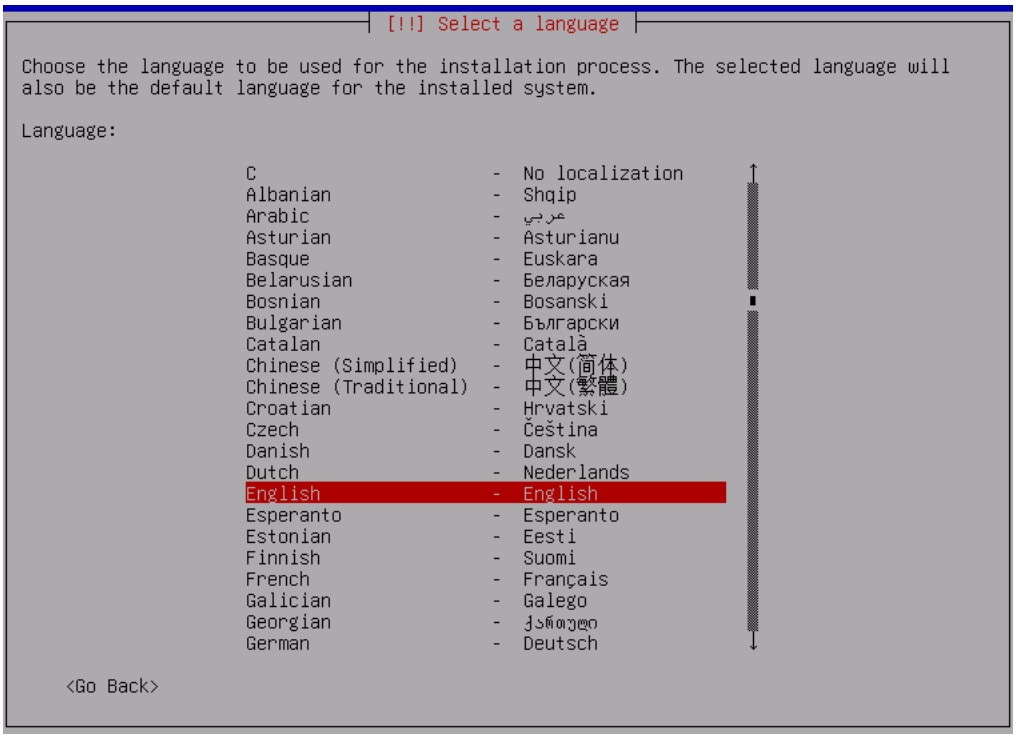


Figure 3 - choose proper language, country, and keyboard language

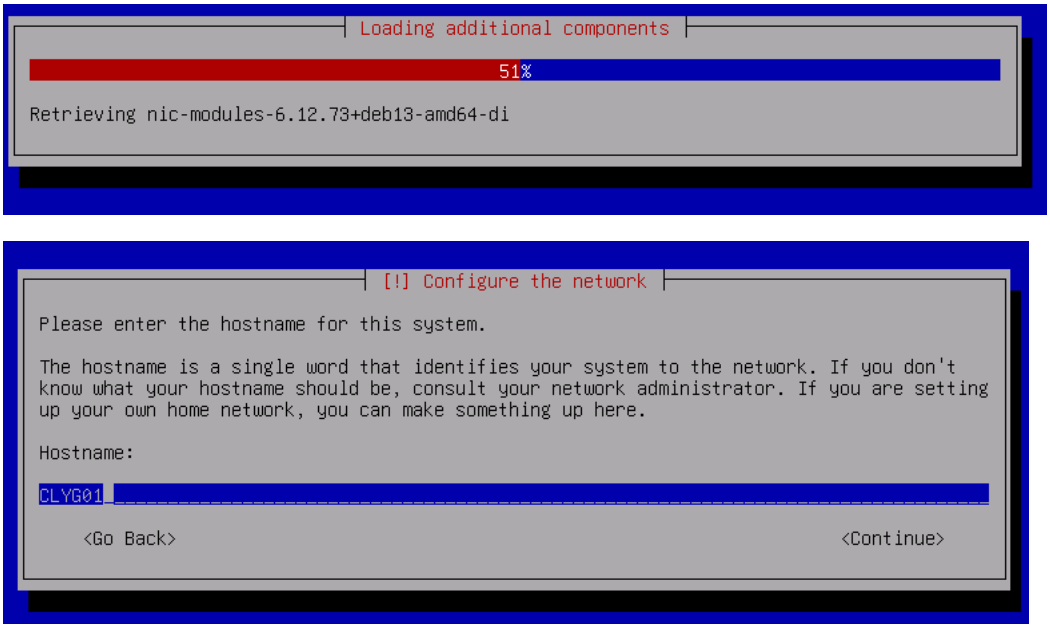


Figure 4 - named the host according to naming conventions

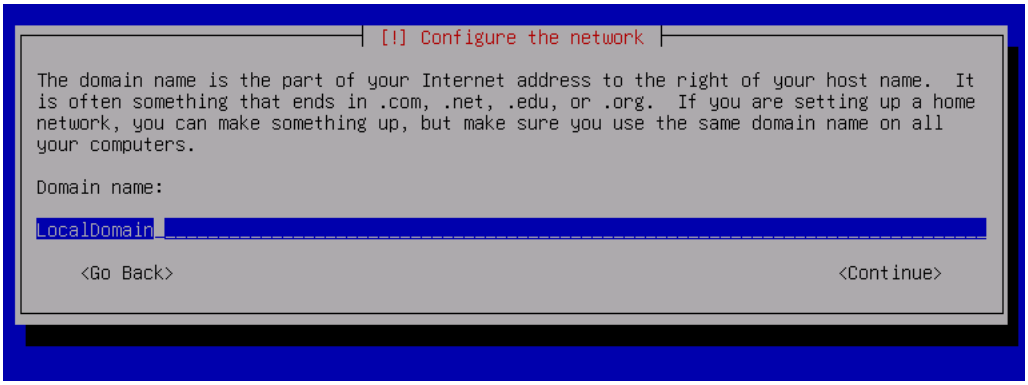


Figure 5 - This section will not be used in this project - can be left as default



Figure 6 - I didn't set a password for root so it is not activated, my user will be administrative and privileged in permissions

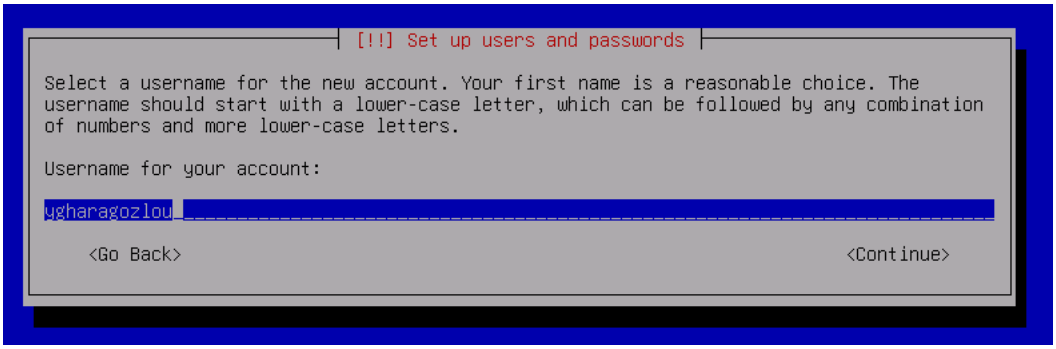


Figure 7 - username per naming conventions, after that set password, correct time zone

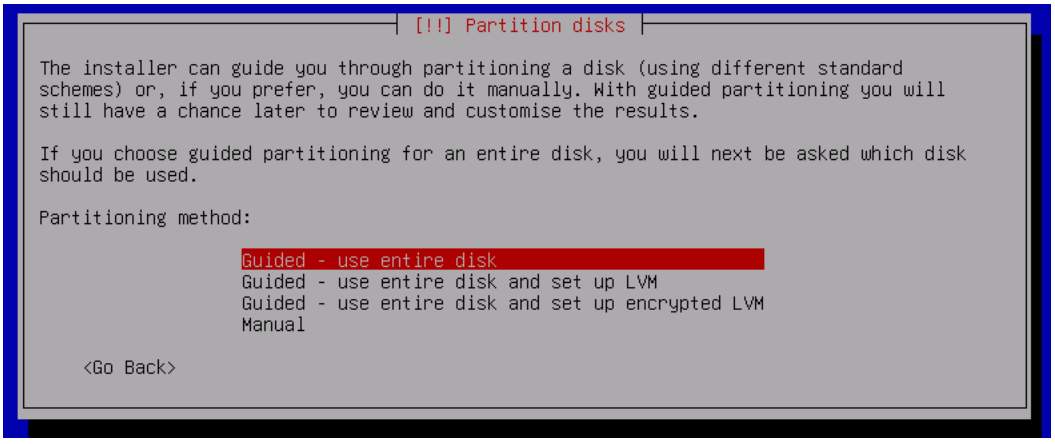


Figure 8 - default

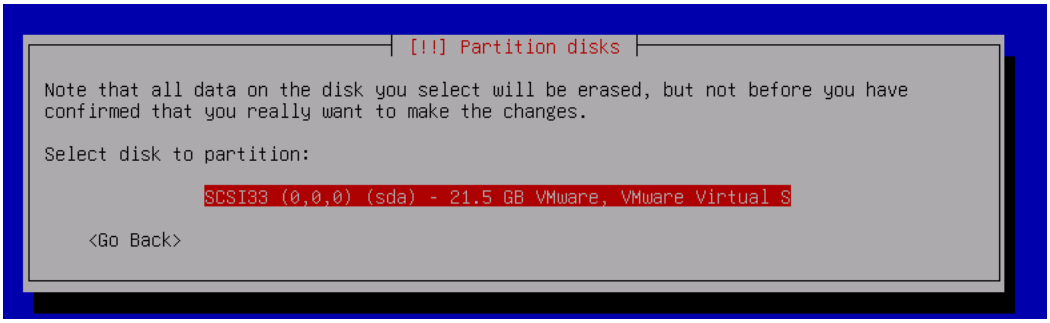


Figure 9 – default

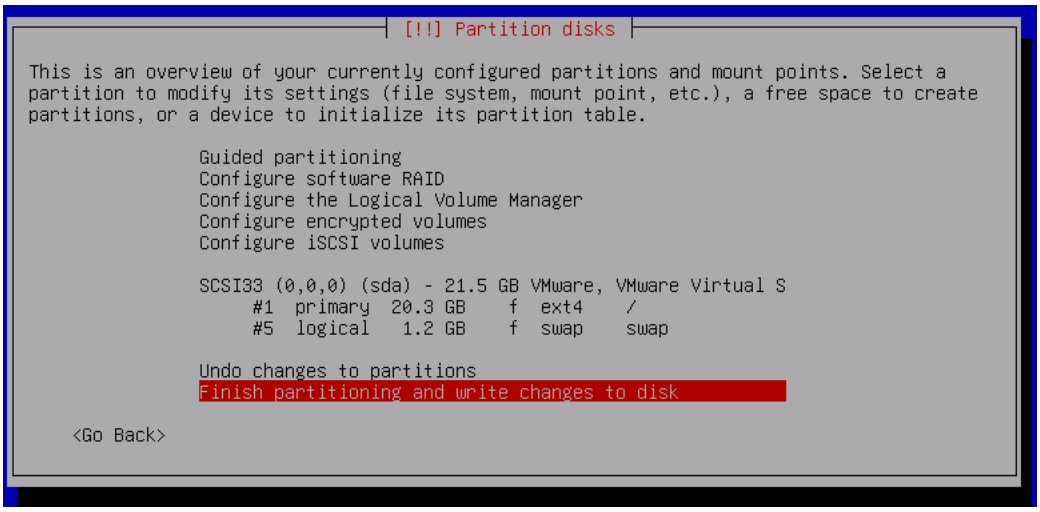
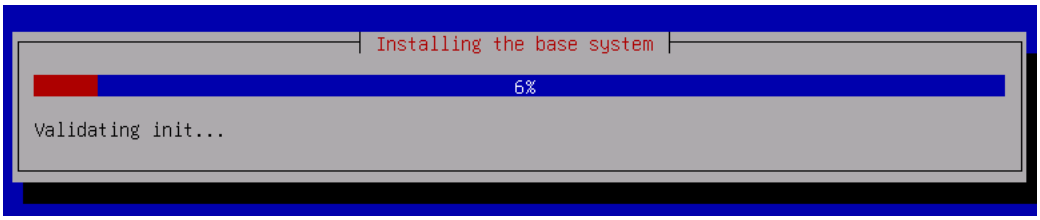


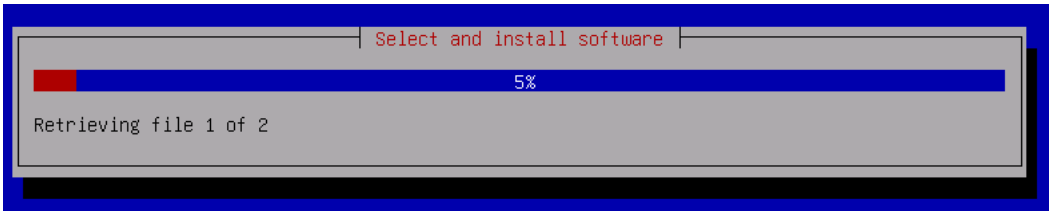
Figure 10 – default

Scan extra installation media? -NO

Participation in package usage survey? -Optional

Note: DO NOT install GUI as this project is on command line only





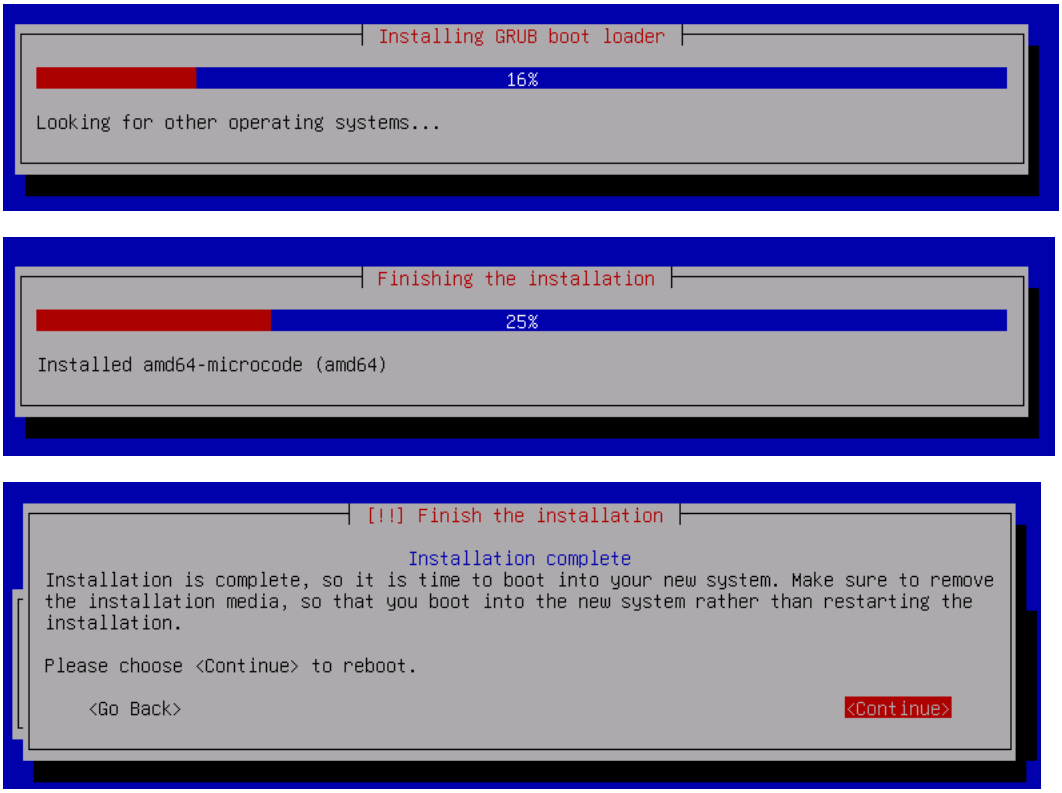


Figure 11 - installation completed.



Figure 12 - this information shows that we are in a minimal environment (see arrows), low CPU usage, low number of installed packages, and the version of distro which is a minimal version.

System updates

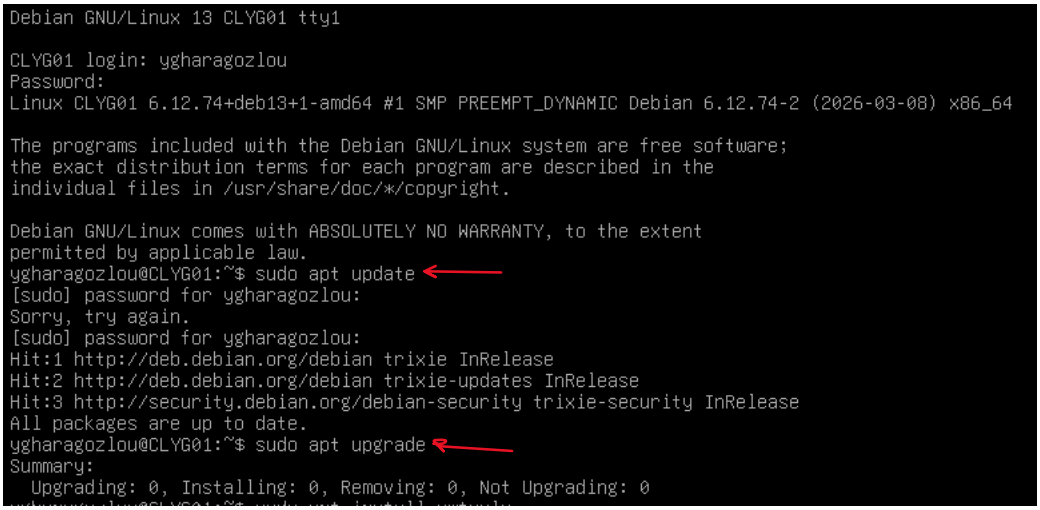


Figure 13 - after installation, boot up and login, perform system updates

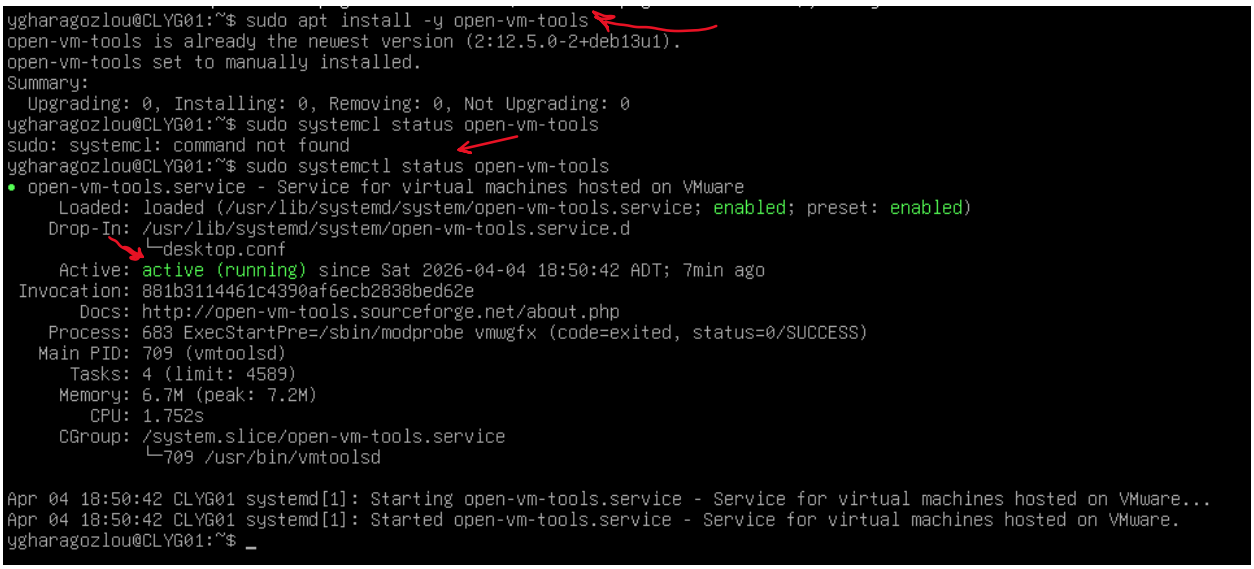


Figure 14 - VM tools

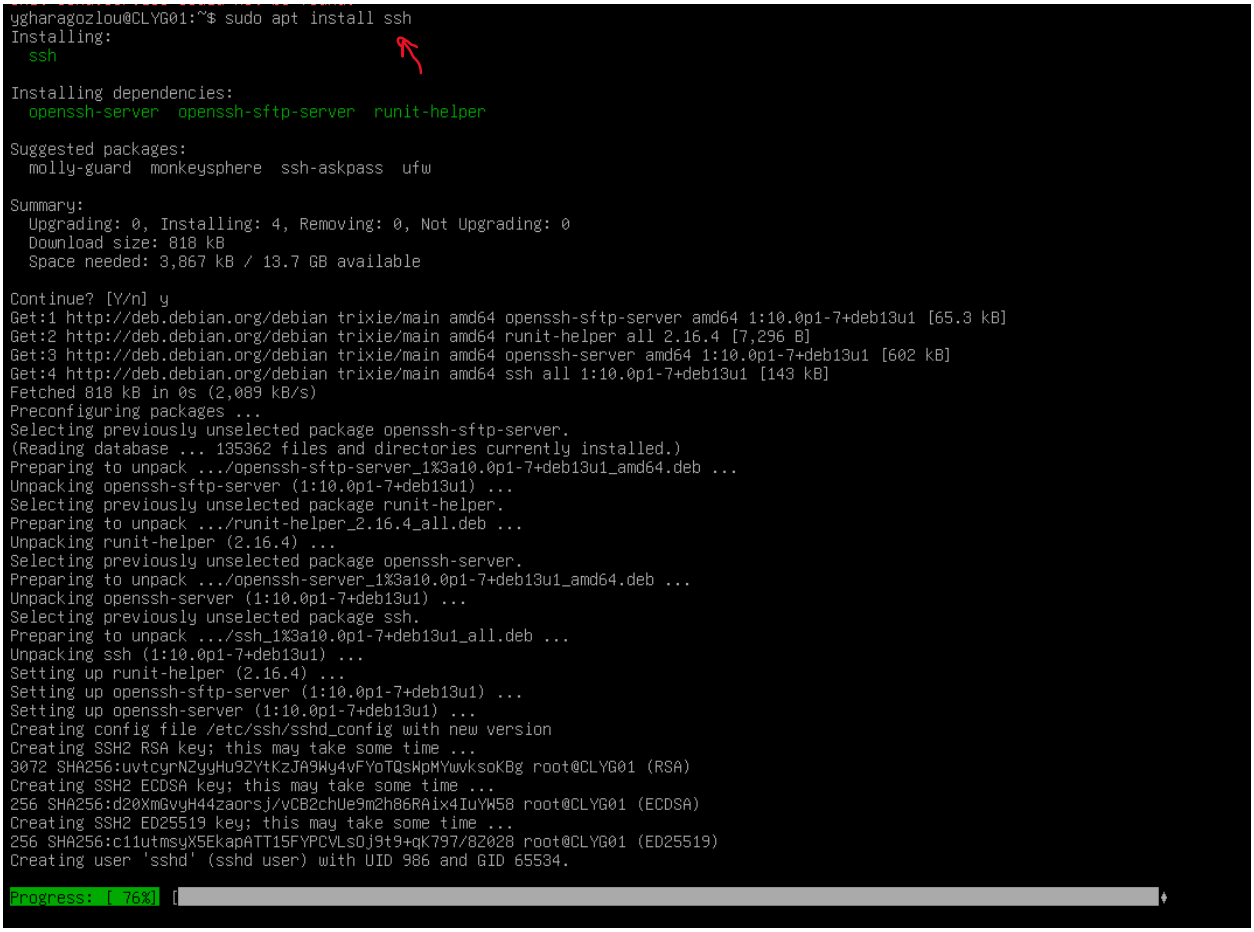


Figure 15 - setting up ssh

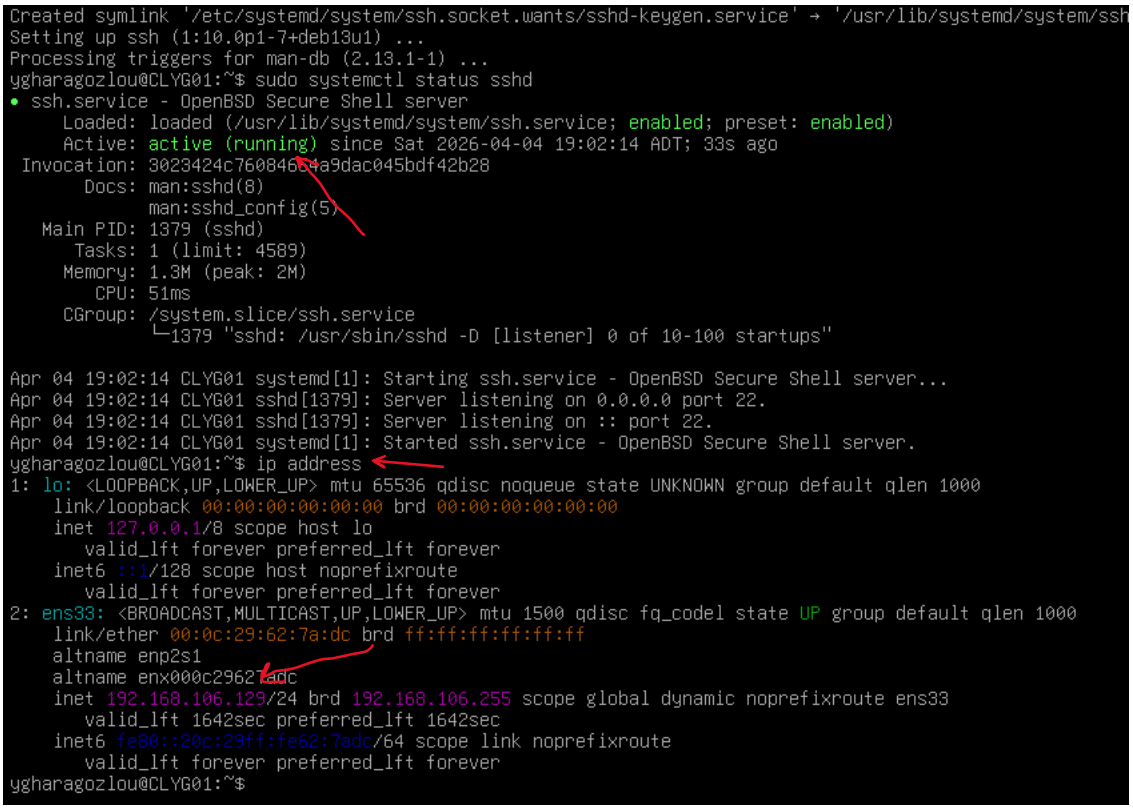


Figure 16 - ssh server running and IP address shown



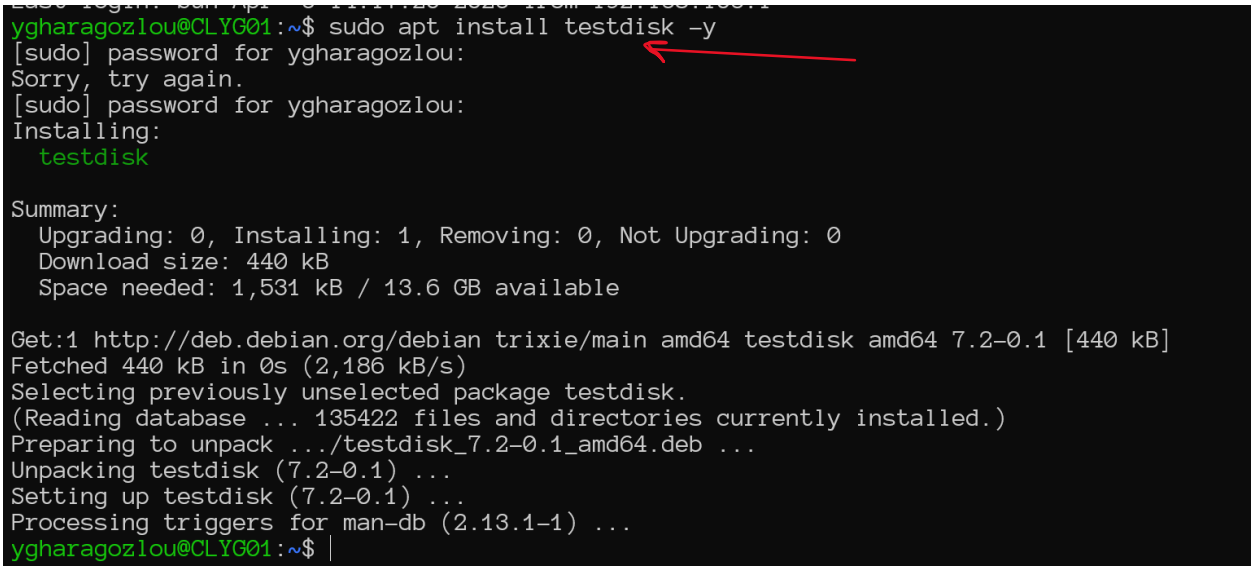
Figure 17 - ssh connection with host machine set and reading from and writing to NTFS checked (ICOP Technology Inc., 2024).

App installation & configuration

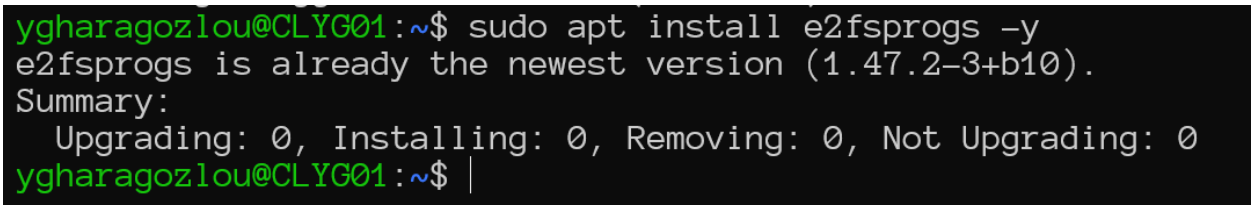
In this section I go through some packages and tools that I downloaded and installed so that I can make the rescue tools/scripts.

util-linux (usually already installed): Provides lsblk, mount, umount and is required for almost every tool.

testdisk: Used in partition repair script. Recovers lost/damaged partition tables:



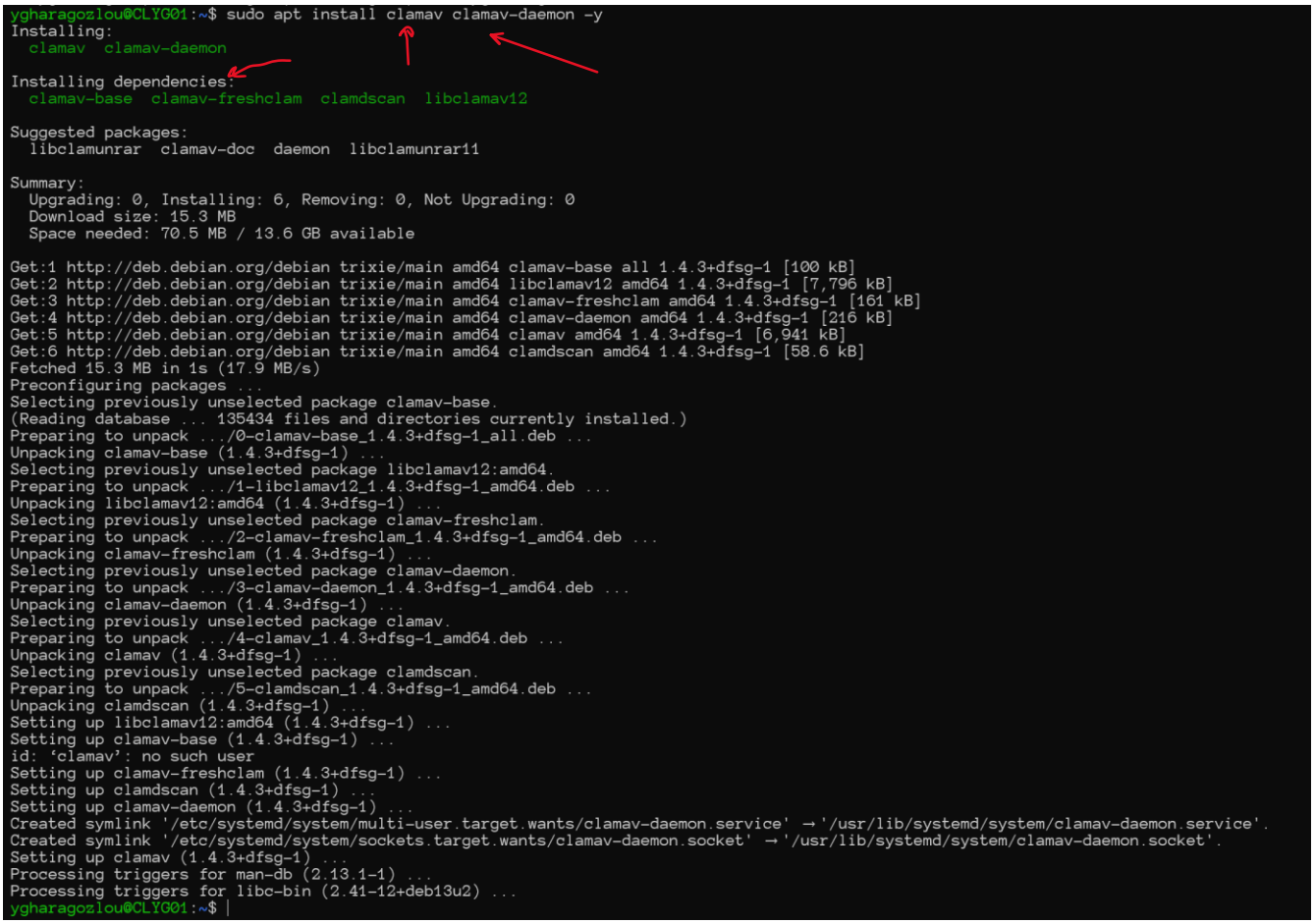
fsck (filesystem check tools): Usually included, but filesystem-specific tools may be needed:



coreutils (already installed): Provides cp (used in recovery script)

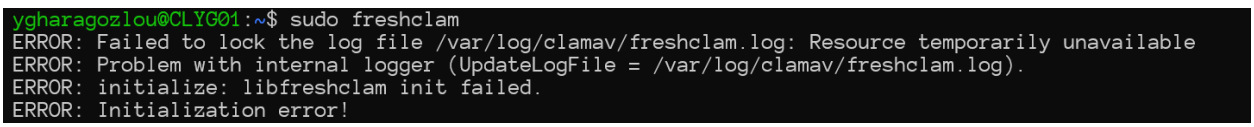
dd (already installed): Used in image backup script

clamav: my antivirus engine:



Its daemon (clamd) acts like a local scanning service.

Before running the antivirus, freshclam should be run, so that it can have updated database:



```
ygharagozlou@CLYG01:~$ systemctl status clamav-freshclam
● clamav-freshclam.service - ClamAV virus database updater
     Loaded: loaded (/usr/lib/systemd/system/clamav-freshclam.service; disabled; preset: enabled)
     Active: active (running) since Sun 2026-04-05 19:02:33 ADT; 4min 23s ago
 Invocation: 2ce592cdd5774d90b0a5a09c4965e6dc
       Docs: man:freshclam(1)
             man:freshclam.conf(5)
             https://docs.clamav.net/
   Main PID: 2011 (freshclam)
      Tasks: 1 (limit: 4589)
     Memory: 117.4M (peak: 674.9M)
        CPU: 19.919s
     CGroup: /system.slice/clamav-freshclam.service
             └─2011 /usr/bin/freshclam -d --foreground=true
ygharagozlou@CLYG01:~$
```

```
ygharagozlou@CLYG01:~$ ls /var/lib/clamav
bytecode.cvd  daily.cvd  freshclam.dat  main.cvd
```

Figure 18 - Virus definitions are fully downloaded. ClamAV is ready to scan offline

SMART: used for monitoring, analyzing, and reporting the health of storage devices (HDD and SSD) to prevent data loss:

```
ygharagozlou@CLYG01:~$ sudo apt install smartmontools -y
[sudo] password for ygharagozlou:
Installing:
  smartmontools

Suggested packages:
  gsmartcontrol  smart-notifier  mailx  | mailutils

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
  Download size: 648 kB
  Space needed: 2,426 kB / 13.5 GB available

Get:1 http://deb.debian.org/debian trixie/main amd64 smartmontools amd64 7.4-3 [648 kB]
Fetched 648 kB in 0s (2,340 kB/s)
Selecting previously unselected package smartmontools.
(Reading database ... 135539 files and directories currently installed.)
Preparing to unpack .../smartmontools_7.4-3_amd64.deb ...
Unpacking smartmontools (7.4-3) ...
Setting up smartmontools (7.4-3) ...
/var/lib/smartmontools/drivedb/drivedb.h 7.3/5528 newly installed (NOT VERIFIED)
Created symlink '/etc/systemd/system/smartd.service' → '/usr/lib/systemd/system/smartmontools.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/smartmontools.service' → '/usr/lib/systemd/system/smartmontools.service'.
Processing triggers for man-db (2.13.1-1) ...
```

```
ygharagozlou@CLYG01:~$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0 18.9G  0 part /
├─sda2   8:2    0    1K  0 part
└─sda5   8:5    0  1.1G  0 part [SWAP]
sr0     11:0    1  754M  0 rom
ygharagozlou@CLYG01:~$ sudo smartctl -H /dev/sda1
smartctl 7.4 2023-08-01 r5530 [x86_64-linux-6.12.74+deb13+1-amd64] (local build)
Copyright (C) 2002-23, Bruce Allen, Christian Franke, www.smartmontools.org

=== START OF READ SMART DATA SECTION ===
SMART Health Status: OK

ygharagozlou@CLYG01:~$
```

Figure 19 - using SMART on a drive

vim and micro

```
yharagozlou@CLYG01:~$ sudo apt install vim
[sudo] password for yharagozlou:
Installing:
  vim

Installing dependencies:
  vim-runtime

Suggested packages:
  ctags  vim-doc  vim-scripts

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
  Download size: 8,797 kB
  Space needed: 43.4 MB / 13.4 GB available

Continue? [Y/n] y
Get:1 http://deb.debian.org/debian trixie/main amd64 vim-runtime all 2:9.1.1230-2 [7,127 kB]
Get:2 http://deb.debian.org/debian trixie/main amd64 vim amd64 2:9.1.1230-2 [1,670 kB]
Fetched 8,797 kB in 0s (17.9 MB/s)
Selecting previously unselected package vim-runtime.
(Reading database ... 135822 files and directories currently installed.)
Preparing to unpack .../vim-runtime_2%3a9.1.1230-2_all.deb ...
Adding 'diversion of /usr/share/vim/vim91/doc/help.txt to /usr/share/vim/vim91/doc/help.txt.vim-tiny by vim-runtime'
Adding 'diversion of /usr/share/vim/vim91/doc/tags to /usr/share/vim/vim91/doc/tags.vim-tiny by vim-runtime'
Unpacking vim-runtime (2:9.1.1230-2) ...
Selecting previously unselected package vim.
Preparing to unpack .../vim_2%3a9.1.1230-2_amd64.deb ...
Unpacking vim (2:9.1.1230-2) ...
Setting up vim-runtime (2:9.1.1230-2) ...
Setting up vim (2:9.1.1230-2) ...
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/ex (ex) in auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/rview (rview) in auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/rvim (rvim) in auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/vi (vi) in auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/view (view) in auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/vim (vim) in auto mode
update-alternatives: using /usr/bin/vim.basic to provide /usr/bin/vimdiff (vimdiff) in auto mode
Processing triggers for man-db (2.13.1-1) ...
Error: Timeout was reached
yharagozlou@CLYG01:~$
```

```
yharagozlou@CLYG01:~$ sudo apt install micro
[sudo] password for yharagozlou:
Installing:
  micro

Installing dependencies:
  xclip

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
  Download size: 4,220 kB
  Space needed: 15.0 MB / 13.5 GB available

Continue? [Y/n] y
Get:1 http://deb.debian.org/debian trixie/main amd64 micro amd64 2.0.14-1+b6 [4,198 kB]
Get:2 http://deb.debian.org/debian trixie/main amd64 xclip amd64 0.13-4 [21.3 kB]
Fetched 4,220 kB in 0s (11.7 MB/s)
Selecting previously unselected package micro.
(Reading database ... 135585 files and directories currently installed.)
Preparing to unpack .../micro_2.0.14-1+b6_amd64.deb ...
Unpacking micro (2.0.14-1+b6) ...
Selecting previously unselected package xclip.
Preparing to unpack .../xclip_0.13-4_amd64.deb ...
Unpacking xclip (0.13-4) ...
Setting up micro (2.0.14-1+b6) ...
Setting up xclip (0.13-4) ...
Processing triggers for desktop-file-utils (0.28-1) ...
Processing triggers for gnome-menus (3.36.0-3) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for mailcap (3.74) ...
```

Hardware information tools: lshw, inxi provide reports of hardware information:

```
yghara gozlou@CLYG01:~$ sudo apt install lshw
Installing:
  lshw

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
  Download size: 312 kB
  Space needed: 994 kB / 13.4 GB available

Get:1 http://deb.debian.org/debian trixie/main amd64 lshw amd64 02.19.git.2021.06.19.996aaad9c7-2.1 [312 kB]
Fetched 312 kB in 0s (1,992 kB/s)
Selecting previously unselected package lshw.
(Reading database ... 135660 files and directories currently installed.)
Preparing to unpack .../lshw_02.19.git.2021.06.19.996aaad9c7-2.1_amd64.deb ...
Unpacking lshw (02.19.git.2021.06.19.996aaad9c7-2.1) ...
Setting up lshw (02.19.git.2021.06.19.996aaad9c7-2.1) ...
Processing triggers for man-db (2.13.1-1) ...
```

```
yghara gozlou@CLYG01:~$ sudo apt install inxi
Installing:
  inxi

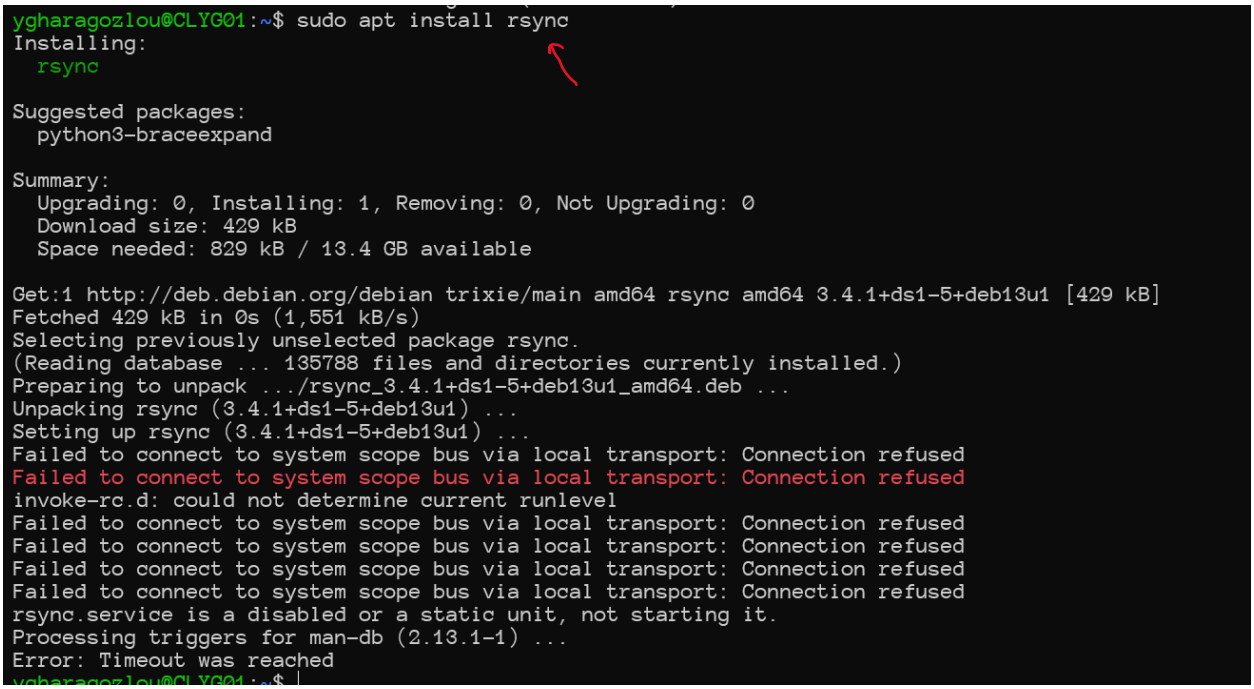
Installing dependencies:
  lm-sensors  mesa-utils  mesa-utils-bin  tree

Suggested packages:
  libxml-dumper-perl  fancontrol  i2c-tools  read-edid

Summary:
  Upgrading: 0, Installing: 5, Removing: 0, Not Upgrading: 0
  Download size: 1,895 kB
  Space needed: 5,456 kB / 13.4 GB available

Continue? [Y/n] y
Get:1 http://deb.debian.org/debian trixie/main amd64 inxi all 3.3.38-1-1 [623 kB]
Get:2 http://deb.debian.org/debian trixie/main amd64 lm-sensors amd64 1:3.6.2-2 [101 kB]
Get:3 http://deb.debian.org/debian trixie/main amd64 mesa-utils-bin amd64 9.0.0-2+b2 [158 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 mesa-utils amd64 9.0.0-2+b2 [953 kB]
Get:5 http://deb.debian.org/debian trixie/main amd64 tree amd64 2.2.1-1 [59.4 kB]
Fetched 1,895 kB in 0s (6,314 kB/s)
Selecting previously unselected package inxi.
(Reading database ... 135667 files and directories currently installed.)
Preparing to unpack .../inxi_3.3.38-1-1_all.deb ...
Unpacking inxi (3.3.38-1-1) ...
Selecting previously unselected package lm-sensors.
Preparing to unpack .../lm-sensors_1%3a3.6.2-2_amd64.deb ...
Unpacking lm-sensors (1:3.6.2-2) ...
Selecting previously unselected package mesa-utils-bin:amd64.
Preparing to unpack .../mesa-utils-bin_9.0.0-2+b2_amd64.deb ...
Unpacking mesa-utils-bin:amd64 (9.0.0-2+b2) ...
Selecting previously unselected package mesa-utils.
Preparing to unpack .../mesa-utils_9.0.0-2+b2_amd64.deb ...
Unpacking mesa-utils (9.0.0-2+b2) ...
Selecting previously unselected package tree.
Preparing to unpack .../tree_2.2.1-1_amd64.deb ...
Unpacking tree (2.2.1-1) ...
Setting up inxi (3.3.38-1-1) ...
Setting up mesa-utils-bin:amd64 (9.0.0-2+b2) ...
Setting up tree (2.2.1-1) ...
Setting up lm-sensors (1:3.6.2-2) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/lm-sensors.service' → '/usr/lib/systemd/system/lm-sensors.service'.
Setting up mesa-utils (9.0.0-2+b2) ...
Processing triggers for man-db (2.13.1-1) ...
```

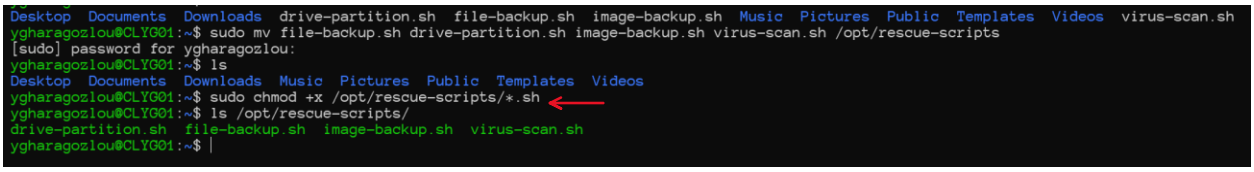
rsync: similar functionality to cp, but with more features and better for larger files

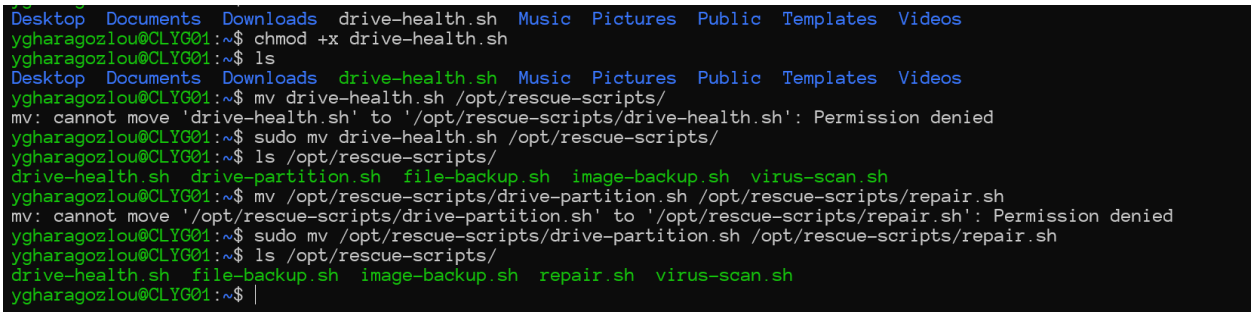


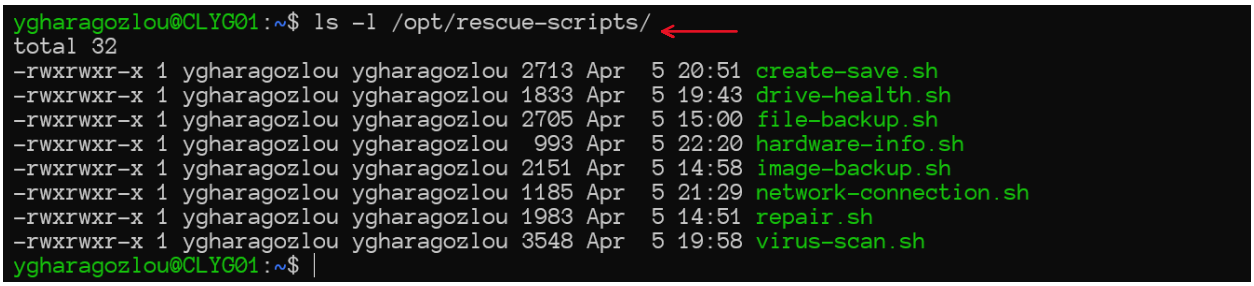
Scripts

The following are steps to create scripts, making them executable, and saving them in their proper locations:

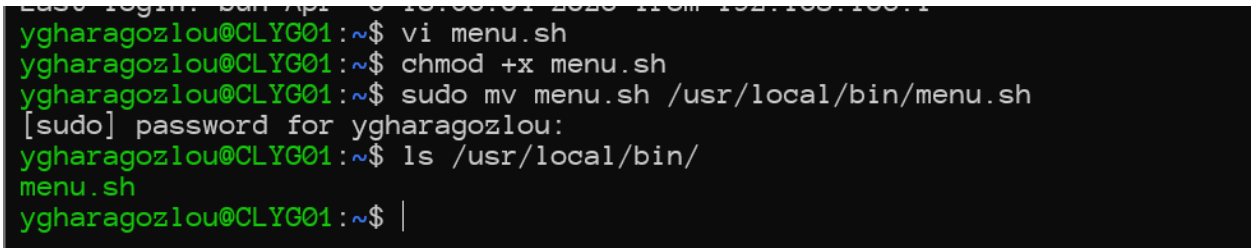








I would like the menu to be a command (so that the user can type “menu.sh” and see the menu appearing on cmd), so I save it here:



Edit user's .bashrc (config file telling the system what files to run after booting up):

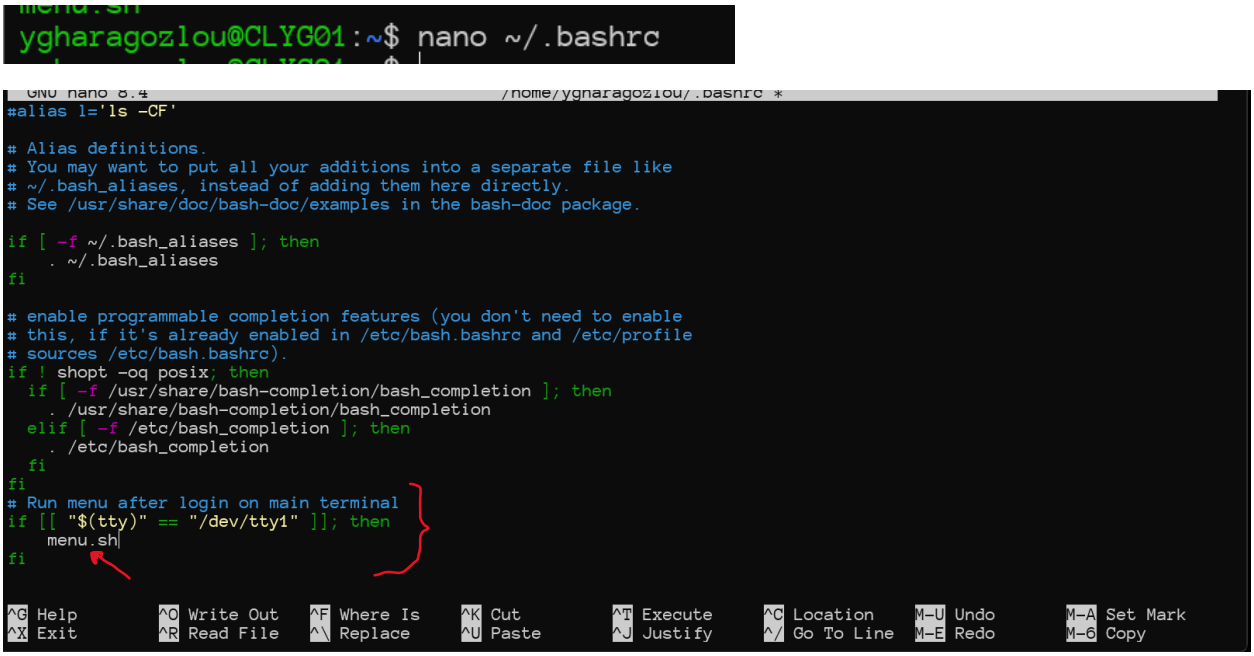


Figure 20 -the section at the bottom was added to .bashrc file (menu.sh is treated like a command because it is saved in /usr/local/bin/)

In the following pages, I paste the scripts (bash). These contain sudo as they are being run by administrative user.

Note: for the purpose of creating a live bootable ISO file later in the project, I had to remove all the 'sudo' keywords in all the scripts, as in the environment of the live ISO, system runs with only the root user.

Menu (menu.sh):

```
#!/bin/bash

#=========================================================
# Author:     Yasaman Gharagozlou
# Created:    2026-04-06
# Description: Menu to show options to the user and ask
#           an input, then run the the related script
#=========================================================
# Infinite loop to keep showing Menu
while true
do
  clear
  echo "___Yasaman_Gharagozlou_____________________W0522396___"
  echo "                  Menu"
  echo "==========================================================="
  echo "1. Drives Health Check (recommended before Repair)"
  echo "2. Repair Partition Table/File System"
  echo "3. Recover Files/Directories"
  echo "4. Image Backup (recommended if a drive is failing)"
  echo "5. Virus Scan (recommended before opening recovered files)"
  echo "6. Text Editor"
  echo "7. Test Netwrok Connectivity"
  echo "8. Hardware Information"
  echo "9. Exit"
  echo "==========================================================="

  # Get user input
  read -p "Choose an option [1-9]: " choice

  # Run related script
  case $choice in
    1)
      echo "Running drive-health.sh script..."
      sleep 3
      /opt/rescue-scripts/drive-health.sh
      ;;
    2)
      echo "Running repair.sh script..."
```

```
    sleep 3
    /opt/rescue-scripts/repair.sh
    ;;
  3)
    echo "Running file-backup.sh script..."
    sleep 3
    /opt/rescue-scripts/file-backup.sh
    ;;
  4)
    echo "Running image-backup.sh script..."
    sleep 3
    /opt/rescue-scripts/image-backup.sh
    ;;
  5)
    echo "Running virus-scan.sh script..."
    sleep 3
    /opt/rescue-scripts/virus-scan.sh
    ;;
  6)
    echo "Running create-save.sh script..."
    sleep 3
    /opt/rescue-scripts/create-save.sh
    ;;
  7)
    echo "Running network-connection.sh..."
    sleep 3
    /opt/rescue-scripts/network-connection.sh
    ;;
  8)
    echo "Running hardware-info.sh script..."
    sleep 3
    /opt/rescue-scripts/hardware-info.sh
    ;;
  9)
    read -p "You can run 'menu.sh' to see Menu again. (Press Enter)"
    break
    ;;
  *)
    echo "Invalid option!"
    ;;
esac

# Pause before showing menu again
echo ""
```

```
  read -p "Press Enter to return to Menu..."
done
```

---

drive-health.sh

```
#!/bin/bash

# ==========================================
# Author:     Yasaman Gharagozlou
# Created:    2026-04-05
# Description: Shows available drives, allows user to select a drive,
#           performs a SMART health check, and gives a clear result
#           with recommended next steps
# ==========================================
echo ""
echo "===== AVAILABLE DRIVES ====="
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT
echo "============================="

read -p "Press Enter to continue..."

# Ask user for drive
echo ""
read -p "Enter FULL drive path (e.g., /dev/sda), choose one that supports SMART tools: "
DRIVE

# Validate input
if [ ! -b "$DRIVE" ]; then
  echo "Invalid device. Exiting."
  exit 1
fi

# Enable SMART (if needed)
echo ""
echo "Preparing SMART check..."
sudo smartctl -s on "$DRIVE"

read -p "Press Enter to run health check..."

# Run SMART check
echo ""
echo "===== DRIVE HEALTH CHECK RESULT ====="
sudo smartctl -H "$DRIVE"
```

```
echo "====================================="

echo ""
echo "===== FEEDBACK
============================================================="

echo "If your drive is healthy, you can proceed with repairing and recovering next."
echo "If your drive is unhealthy, create an image backup before anything else."

echo
"=============================================================================
========"

#Final pause
read -p "Press Enter to exit..."
```

file-backup.sh

```
#!/bin/bash

# ============================================
# Author:     Yasaman Gharagozlou
# Created:    2026-04-05
# Description: Shows available drives, mounts a selected drive,
#           allows user to recover specific files/folders
#           to a destination drive
# ============================================
echo ""
echo "===== Available Drives ====="
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINT
echo "============================"

read -p "Press Enter to continue..."

# Ask user for source drive
read -p "Enter FULL path of the drive/partition to mount (e.g., /dev/sda1)" SOURCE_DRIVE

# Create mount point
MOUNT_POINT="/mnt/recovery_source"
sudo mkdir -p $MOUNT_POINT

# Mount the drive
echo "Mounting $SOURCE_DRIVE..."
sudo mount $SOURCE_DRIVE $MOUNT_POINT

if [ $? -ne 0 ]; then
   echo "Mount failed. Exiting."
   exit 1
fi

echo "Mounted successfully at $MOUNT_POINT"

read -p "Press Enter to view contents..."

# Show user first-level directories
echo "===== Top-level contents ====="
sudo ls -ah $MOUNT_POINT
```

```
echo "============================="

read -p "Press Enter to continue..."

# Ask what to recover
read -p "Enter FULL path of file/directory to recover (e.g.,
/mnt/recovery_source/Documents/file.txt): " RECOVER_PATH

# Check if path exists
if [ ! -e "$RECOVER_PATH" ]; then
  echo "Path does not exist. Exiting."
  exit 1
fi

# Ask destination drive
echo "===== Available Drives (Destination) ====="
sudo lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINT
echo "========================================="

read -p "Enter FULL path of destination drive/partition (e.g., /dev/sdb1)" DEST_DRIVE

# Create mount point for destination
DEST_MOUNT="/mnt/recovery_dest"
sudo mkdir -p $DEST_MOUNT

# Mount destination
echo "Mounting destination..."
sudo mount $DEST_DRIVE $DEST_MOUNT

if [ $? -ne 0 ]; then
  echo "Destination mount failed. Exiting."
  exit 1
fi

echo "Mounted destination at $DEST_MOUNT"

read -p "Press Enter to start recovery..."

# Copy (Recover) data
echo "Copying data..."
sudo rsync -av --ignore-errors "$RECOVER_PATH" "$DEST_MOUNT/"
echo ""
echo "Operation completed (Some files may have been skipped due to filesystem
limitations.)"
```

```
echo ""

# Unmount and cleanup
read -p "Press Enter for unmounting drives and cleaning up..."
sudo umount $MOUNT_POINT
sudo umount $DEST_MOUNT
sudo rmdir $MOUNT_POINT
sudo rmdir $DEST_MOUNT
echo "Done."
```

repair.sh

```
#!/bin/bash
# ==========================================
# Author:     Yasaman Gharagozlou
# Created:    2026-04-05
# Description: Shows all drives and partitions,
#           allows partition table or file system repair
# ==========================================
# Show all drives and partitions
echo ""
echo "=== DRIVES & PARTITIONS ==="
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT
echo ""

# Pause so user can read the disk list
read -p "Press Enter to continue..."

# Show repair options
echo ""
echo "Choose an option:"
echo "1) Repair Partition Table (uses testdisk)"
echo "2) Repair File System (uses fsck)"
echo ""

# Get user's choice
read -p "Enter choice (1 or 2): " choice

# Ask for the target device (example: /dev/sda)
echo ""
read -p "Enter full device path (e.g. /dev/sda), DO NOT Enter drives currently in-use: " dev

# Handle user choice
case $choice in

 1)
  # Partition Table Repair Section
  # Uses testdisk to recover partitions
  echo ""
  read -p "Launching TestDisk for $dev... (Press Enter)"
  sudo testdisk "$dev"
  ;;
```

```
 2)
  # File System Repair Section
  # Runs fsck on each partition
  echo ""
  read -p "Scanning partitions on $dev... (Press Enter)"

  # Loop through all partitions of the disk
  for p in $(sudo lsblk -ln -o NAME "$dev" | tail -n +2); do
   echo ""
   echo "Repairing /dev/$p ..."

   # Run file system check and auto-fix
   sudo fsck -y "/dev/$p"

   # Pause after each partition repair
   read -p "Press Enter to continue to next partition..."
  done
  ;;

 *)
  # Invalid input handling
  echo "Invalid option selected."
  ;;
esac

# Final pause so user can see results
echo ""
echo "=== OPERATION COMPLETE ==="
read -p "Press Enter to exit..."
```

image-backup.sh

```
#!/bin/bash

# ==========================================
# Author:     Yasaman Gharagozlou
# Created:    2026-04-05
# Description: Shows available drives, allows the user to select a drive,
#           creates a full disk/partition image backup to a destination drive,
#           and displays progress
# ==========================================
echo ""
echo "===== Available Drives ====="
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINT
echo "============================"

read -p "Press Enter to continue..."

#Ask user for source drive
read -p "Enter FULL path of drive/partition to back up (e.g., /dev/sda): " SOURCE_DRIVE

# Ask for destination drive
echo "===== Available Drives (Destination) ====="
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINT
echo "==========================================="
read -p "Enter FULL path of destination drive (e.g., /dev/sdb1). " DEST_DRIVE

# Mount destination drive
DEST_MOUNT="/mnt/backup_dest"
sudo mkdir -p $DEST_MOUNT

echo "Mounting destination..."
sudo mount $DEST_DRIVE $DEST_MOUNT

if [ $? -ne 0 ]; then
   echo "Failed to mount destination. Exiting."
   exit 1
fi

echo "Mounted at $DEST_MOUNT"

read -p "Press Enter to continue..."
```

```
# Ask for image file name
read -p "Enter name for backup image (e.g., backup.img): " IMAGE_NAME

IMAGE_PATH="$DEST_MOUNT/$IMAGE_NAME"

# Create image backup
echo "Starting image backup..."
echo "This may take a while..."

# Using dd with progress
sudo dd if=$SOURCE_DRIVE of=$IMAGE_PATH bs=4M status=progress | gzip >
"$IMAGE_PATH.gz"

if [ $? -eq 0 ]; then
  echo "Image backup completed successfully!"
  echo "Saved at: $IMAGE_PATH"
else
  echo "Backup failed."
fi

read -p "Press Enter to finish..."

# Cleanup
echo "Unnmounting and clean up..."
sudo umount $DEST_MOUNT
sudo rmdir $DEST_MOUNT
echo "Done."
```

virus-scan.sh

```
#!/bin/bash

#
================================================================================
====
# Author:     Yasaman Gharagozlou
# Created:    2026-04-05
# Description: Shows available drives, asks for user input and mounts the
#           selected drive, scans files for malware, shows infected files,
#           and optionally deletes them
#
================================================================================
====
echo ""
echo "===== AVAILABLE DRIVES ====="
lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINT
echo "============================"

read -p "Press Enter to continue..."

# Ask for destination  drive
echo ""
read -p "Enter FULL path of destination drive (e.g., /dev/sdb1), DO NOT enter an already
mounted drive: " DEST_DRIVE

# Create mount point
MOUNT_POINT="/mnt/recovery_dest"
sudo mkdir -p $MOUNT_POINT

# Mount the drive
echo ""
echo "Mounting $DEST_DRIVE ..."
sudo mount $DEST_DRIVE $MOUNT_POINT

if [ $? -ne 0 ]; then
   echo "Mount failed. Exiting."
   exit 1
fi

echo "Mounted at $MOUNT_POINT"
echo "Top level directories and files"
```

```
sudo ls -ah "$MOUNT_POINT"
read -p "Press Enter to continue..."

# Ask for exact recovery folder (optional)
echo ""
read -p "Enter folder inside drive to scan (or press Enter to scan all): " SUB_PATH

if [ -z "$SUB_PATH" ]; then
  TARGET_PATH="$MOUNT_POINT"
else
  TARGET_PATH="$MOUNT_POINT/$SUB_PATH"
fi

# update virus database
echo ""
echo "Updating virus database (this can take a few seconds)..."
if systemctl is-active --quiet clamav-freshclam; then
  echo "Update service already running, skipping manual update."
else
  sudo freshclam
fi
read -p "Press Enter to start scanning..."

#scan for malware
echo ""
echo "Scanning for malware..."

INFECTED_LIST="/tmp/infected_files.txt"

clamscan -r --infected "$TARGET_PATH" | tee "$INFECTED_LIST"

# Check if infections found
if ! grep -q "FOUND" "$INFECTED_LIST"; then
  echo ""
  echo "No infected files found."
  echo "It is safe to use files on this drive."

  read -p "Press Enter to unmount and clean up..."
  sudo umount $MOUNT_POINT
  sudo rmdir $MOUNT_POINT
  exit 0
fi

# Show infected files
```

```
echo ""
echo "===== INFECTED FILES ====="
grep "FOUND" "$INFECTED_LIST"
echo "=========================="

read -p "Press Enter to continue..."

# Ask user to delete infected files
echo ""
read -p "Do you want to DELETE all infected files? (y/n): " choice

if [[ "$choice" == "y" || "$choice" == "Y" ]]; then

  echo ""
  echo "Deleting infected files..."

  grep "FOUND" "$INFECTED_LIST" | cut -d: -f1 | while read file; do
    sudo rm -f "$file"
    echo "Deleted: $file"
  done

  echo ""
  echo "All infected files have been removed."
else
  echo ""
  echo "No files were deleted."
fi

# cleanup
echo ""
echo "Unmounting and cleaning up..."
sudo umount $MOUNT_POINT
sudo rmdir $MOUNT_POINT
echo "Scan complete. It is now safer to use files on this drive."
read -p "Press Enter to exit..."
```

create-save.sh

```
#!/bin/bash
# ============================================================
# Author:     Yasaman Gharagozlou
# Created:    2026-04-05
# Description: Shows available Text Editors and lets user choose
#           which one to open.
# ============================================================
echo ""
echo "=== Text Editors ==="
# Choose editor
echo ""
echo "Choose editor:"
echo "1. nano"
echo "2. vim"
echo "3. micro"
read -p "Enter choice [1-3]: " choice

case $choice in
  1)
    editor="nano"
    echo "Opening nano..."
    sleep 2
    ;;
  2)
    editor="vim"
    echo "Opening vim..."
    sleep 2
    ;;
  3)
    editor="micro"
    echo "Opening micro..."
    sleep 2
    ;;

  *)
    #Invalid choices
    echo "Invalid option."
    exit 1
esac

# Open selected editor
```

$editor

network-connection.sh

```
#!/bin/bash

#============================================================
# Author: Yasaman Gharagozlou
# Created: 2026-04-05
# Description: Network connectivity test providing a report.
#============================================================
echo ""
echo "=== Network Connectivity Test ==="

# Test loopback interface
echo "Testing loopback at 127.0.0.1... "
if ping -c 4 127.0.0.1 &> /dev/null; then
  echo "OK"
else
  echo "Failed"
fi

# Test local network connectivity (default gateway)
GATEWAY=$(ip route | grep default | awk '{print $3}')
if [ -n "$GATEWAY" ]; then
  echo "Pinging local gateway $GATEWAY... "
  if ping -c 4 $GATEWAY &> /dev/null; then
    echo "Reachable"
  else
    echo "Cannot reach gateway"
  fi
else
  echo "No gateway found"
fi

# Testing internet connectivity by pinging Google DNS
echo "Pinging 8.8.8.8 (Google DNS Server)..."
if ping -c 4 8.8.8.8 &> /dev/null; then
  echo "Online"
else
  echo "Cannot reach 8.8.8.8"
fi

# Test internet connectivity by resolving google.com
echo "Resolving google.com... "
```

```
if nslookup google.com &> /dev/null; then
  echo "Done"
else
   echo "Cannot resolve Google domain name"
fi

echo "=== Test Complete ==="
```

hardware-info.sh

```
#!/bin/bash

#=============================================================================
=
# Author: Yasaman Gharagozlou
# Created: 2026-04-05
# Description: System information report are shown with optional details.
#=============================================================================
=
echo "=== System Information ==="

# Full overview with inxi
echo "General system overview:"
inxi -Fxz | less

# Ask if user wants detailed hardware info
read -p "Show detailed hardware info (lshw -short)? [y/N] " ans
if [[ "$ans" = "y" || "$ans" = "Y" ]]; then
  sudo lshw -short | less
fi

# Ask for CPU info
read -p "Show CPU info (lscpu)? [y/N] " ans
if [[ "$ans" = "y" || "$ans" = "Y" ]]; then
  lscpu | less
fi

# Ask for memory info
read -p "Show memory info (free -h)? [y/N] " ans
if [[ "$ans" = "y" || "$ans" = "Y" ]]; then
  free -h
fi

# Ask for storage info
read -p "Show block devices (lsblk)? [y/N] " ans
if [[ "$ans" = "y" || "$ans" = "Y" ]]; then
   lsblk
   read -p "Press Enter to continue..."
fi

echo "=== System Info Check Complete ==="
```

Creating a bootable ISO

There are different methods to create an ISO image from a system. I primarily tried creating a live ISO using live-build (special tools for Debian), but due to difficult configuration and set up, all my attempts had errors and were not configured properly. Only one of my attempts was functional, which had full GUI set up and was not appropriate for the purpose of this assignment (Debian Install Notes, 2020).

Creating a clone of the system and turning it into an ISO was another method that I tried. This method is very unlikely to work, because it creates an unstable and often incompatible ISO file, and it is very hard to add bootloader to it properly. So, in most cases, the resulting ISO image is not properly bootable, let alone being a live system that would run in RAM.

Finally, the third method was to download and extract a Debian minimal (command line only) live ISO image, add my scripts, configurations, and packages to it, and repackaging it to an ISO again. Then making it bootable. This method worked to some extent.

The following are the steps I took in this method.

Method 3:

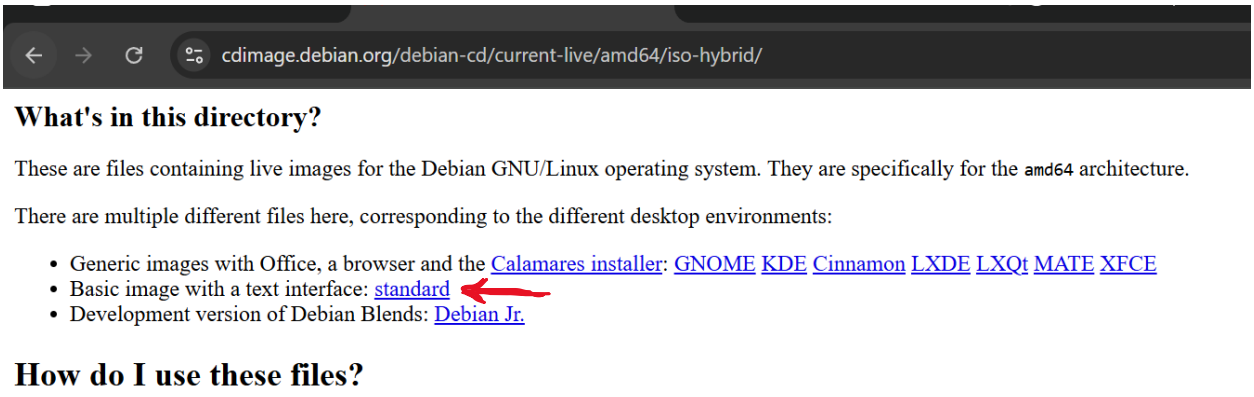


Figure 21 - link to download live ISO

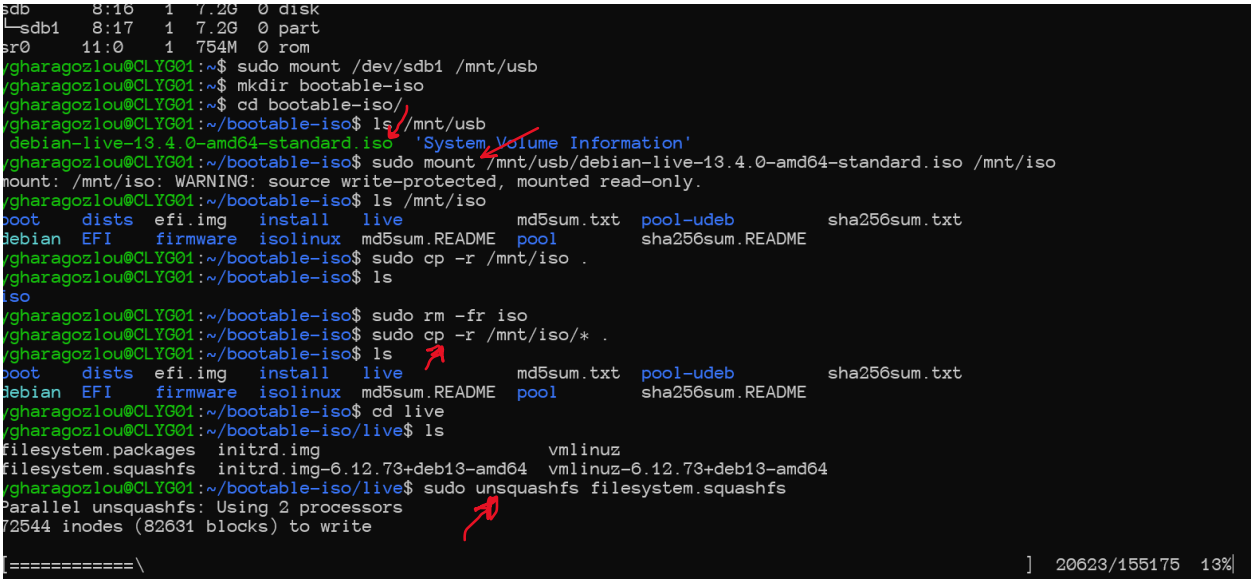


Figure 22 - copied the live ISO from USB to VM, mounted ISO, copied its content to a directory in my user home, unpacked it's file system (which contains the root of the system)

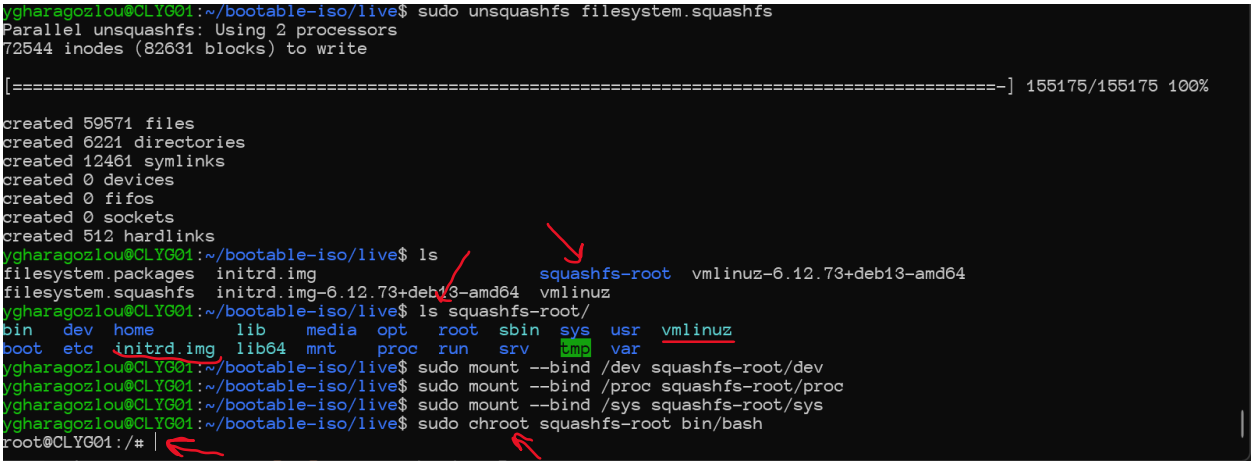


Figure 23 – navigated to the root of the system, prepared the environment for using chroot, and logged into chroot (the root user in the system)

Using this command, added all of my required packages to the system (after making sure system is online):

apt install -y util-linux testdisk e2fsprogs coreutils clamav clamav-daemon smartmontools rsync lshw inxi vim micro procps

```
ocale: Cannot set LC_MESSAGES to default locale: No such file or directory
ocale: Cannot set LC_ALL to default locale: No such file or directory
unning in chroot, ignoring request.
etting up clamdscan (1.4.3+dfsg-1) ...
etting up clamav-daemon (1.4.3+dfsg-1) ...
ocale: Cannot set LC_CTYPE to default locale: No such file or directory
ocale: Cannot set LC_MESSAGES to default locale: No such file or directory
ocale: Cannot set LC_ALL to default locale: No such file or directory
unning in chroot, ignoring request.
reated symlink '/etc/systemd/system/multi-user.target.wants/clamav-daemon.service' → '/usr/lib/systemd/system/clamav-da
mon.service'.
reated symlink '/etc/systemd/system/sockets.target.wants/clamav-daemon.socket' → '/usr/lib/systemd/system/clamav-daemon
socket'.
etting up ntfs-3g (1:2022.10.3-5) ...
etting up clamav (1.4.3+dfsg-1) ...
etting up testdisk (7.2-0.1) ...
rocessing triggers for initramfs-tools (0.148.3) ...
pdate-initramfs: Generating /boot/initrd.img-6.12.73+deb13-amd64
: mkconf: MD subsystem is not loaded, thus I cannot scan for arrays.
: mdadm: failed to auto-generate temporary mdadm.conf file.
ive-boot: core filesystems dm-verity devices utils udev blockdev dns.
rocessing triggers for libc-bin (2.41-12+deb13u2) ...
rocessing triggers for man-db (2.13.1-1) ...
rocessing triggers for mailcap (3.74) ...
oot@CLYG01:/#
```

```
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for mailcap (3.74) ...
root@CLYG01:/# exit
exit
PING debian.org (151.101.66.132) 56(84) bytes of data.
64 bytes from 151.101.66.132: icmp_seq=1 ttl=128 time=18.2 ms
64 bytes from 151.101.66.132: icmp_seq=2 ttl=128 time=17.8 ms
64 bytes from 151.101.66.132: icmp_seq=3 ttl=128 time=19.4 ms

--- debian.org ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 17.786/18.444/19.379/0.679 ms
ygharagozlou@CLYG01:~/bootable-iso/live$ sudo mkdir -p squashfs-root/opt/rescue-scripts
ygharagozlou@CLYG01:~/bootable-iso/live$ sudo cp /opt/rescue-scripts/* squashfs-root/opt/rescue-scripts/
ygharagozlou@CLYG01:~/bootable-iso/live$ ls  squashfs-root/opt/rescue-scripts/
create-save.sh   file-backup.sh    image-backup.sh        repair.sh
drive-health.sh  hardware-info.sh  network-connection.sh  virus-scan.sh
ygharagozlou@CLYG01:~/bootable-iso/live$ sudo mkdir -p squashfs-root/usr/local/bin
sudo cp /usr/local/bin/menu.sh squashfs-root/usr/local/bin/
ygharagozlou@CLYG01:~/bootable-iso/live$ ls  squashfs-root/usr/local/bin/
menu.sh
ygharagozlou@CLYG01:~/bootable-iso/live$
```

Figure 24 - copied all my scrips to their proper directories inside the system

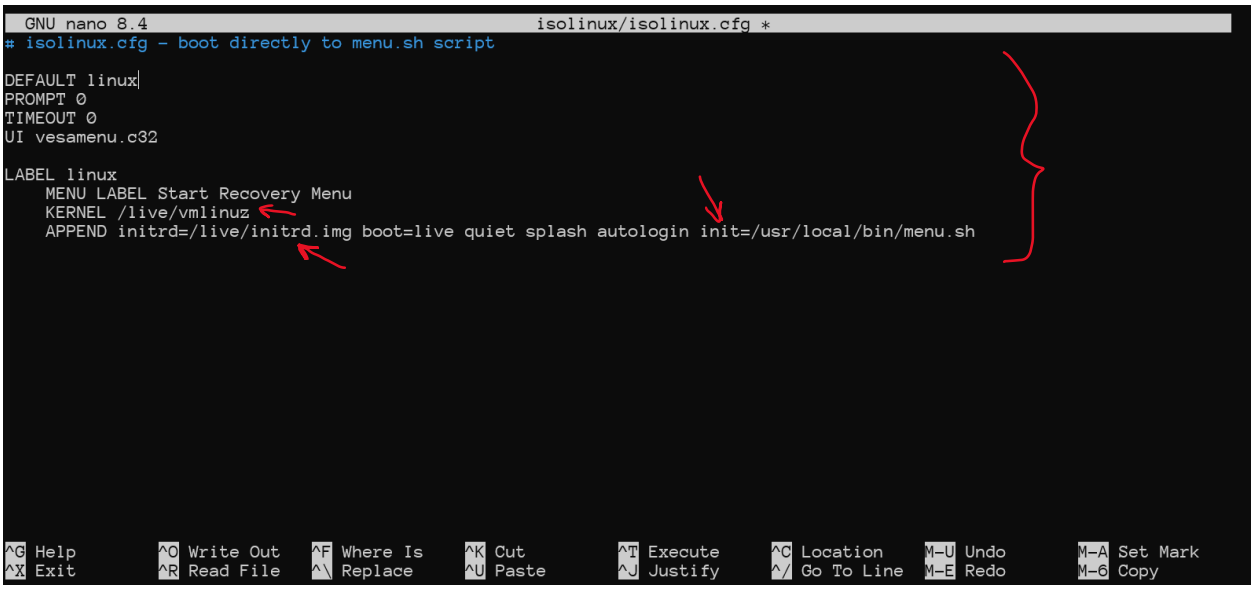


Figure 25 - edited this config file to point to the correct directions at the boot time (vmlinuz, initrd.img, and menu.sh) so that the boot happens properly and the menu appears right after (Red Hat, Inc., n.d.).

Inside chroot I added required packagesconfig files, tested and modified my scripts (just deleted sudo from every script), now I exit and start repacking the ISO (Linux Kernel Documentation, n.d.):



The next step is to rebuild the ISO so it's bootable again. Here's a safe method using xorriso (Rocky Linux Documentation, n.d.):

```
ygharagozlou@CLYG01:~/bootable-iso$ sudo xorriso -as mkisofs \
  -iso-level 3 \
  -full-iso9660-filenames \
  -volid "CUSTOM-RESCUE" \
  -eltorito-boot isolinux/isolinux.bin \
  -eltorito-catalog isolinux/boot.cat \
  -no-emul-boot \
  -boot-load-size 4 \
  -boot-info-table \
  -isohybrid-mbr /usr/lib/ISOLINUX/isohdpfx.bin \
  -eltorito-alt-boot \
  -e EFI/efiboot.img \
  -no-emul-boot \
  -isohybrid-gpt-basdat \
  -output ../custom-rescue.iso \
  .
xorriso : UPDATE :  93.11% done, estimate finish Wed Apr 08 11:28:27 2026
xorriso : UPDATE :  95.36% done, estimate finish Wed Apr 08 11:28:25 2026
xorriso : UPDATE :  97.49% done, estimate finish Wed Apr 08 11:28:24 2026
xorriso : UPDATE :  99.58% done
ISO image produced: 3087504 sectors
Written to medium : 3087504 sectors at LBA 0
Writing to 'stdio:../custom-rescue.iso' completed successfully.

ygharagozlou@CLYG01:~/bootable-iso$ ls
boot    dists  firmware  isolinux  md5sum.README  pool       sha256sum.README
debian  EFI    install   live      md5sum.txt     pool-udeb  sha256sum.txt
ygharagozlou@CLYG01:~/bootable-iso$ cd ..
ygharagozlou@CLYG01:~$ ls
bootable-iso    Desktop    Downloads  newfile.txt  Public     Videos
custom-rescue.iso  Documents  Music      Pictures     Templates  whatever
ygharagozlou@CLYG01:~$
```

Figure 26 - ISO file created.

```
ygharagozlou@CLYG01:~$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0 18.9G  0 part /
├─sda2   8:2    0    1K  0 part
└─sda5   8:5    0  1.1G  0 part [SWAP]
sdb      8:16   1 58.6G  0 disk
sr0     11:0    1  754M  0 rom
ygharagozlou@CLYG01:~$ sudo dd if=~/custom-rescue.iso of=/dev/sdb bs=4M status=progress oflag=sync
184549376 bytes (185 MB, 176 MiB) copied, 9 s, 20.5 MB/s
```

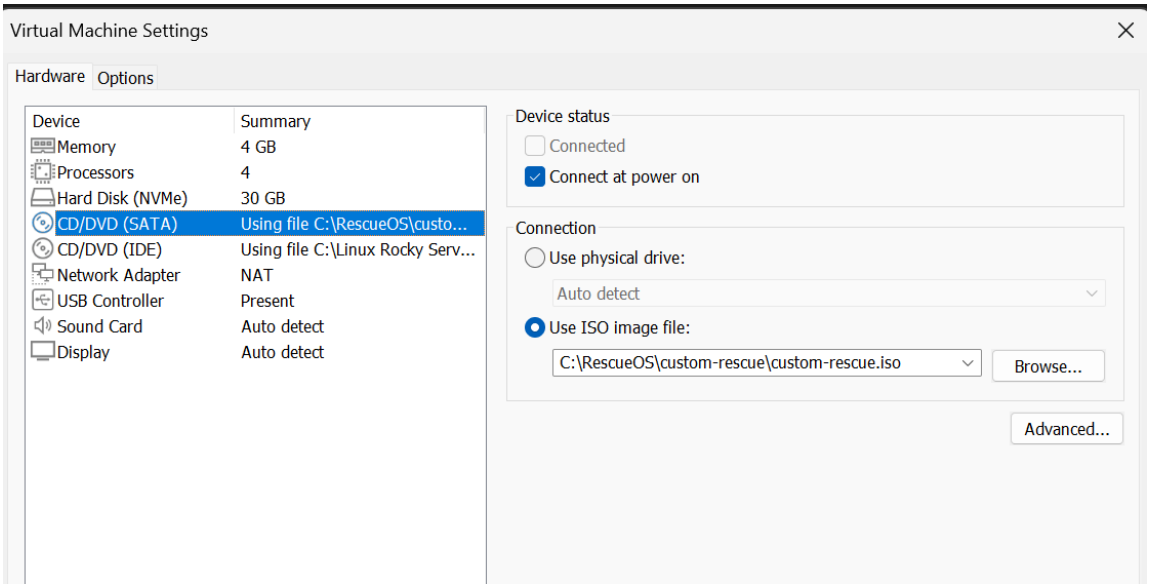
Figure 27 - I tried using dd to write to a usb stick, but when I tested it, it was not bootable. So I used rsync to safely copy the large .iso file to a USB and boot it up as a CD/DVD in a VM.

Project functionality demo

To properly boot from an ISO file, I used these resources:

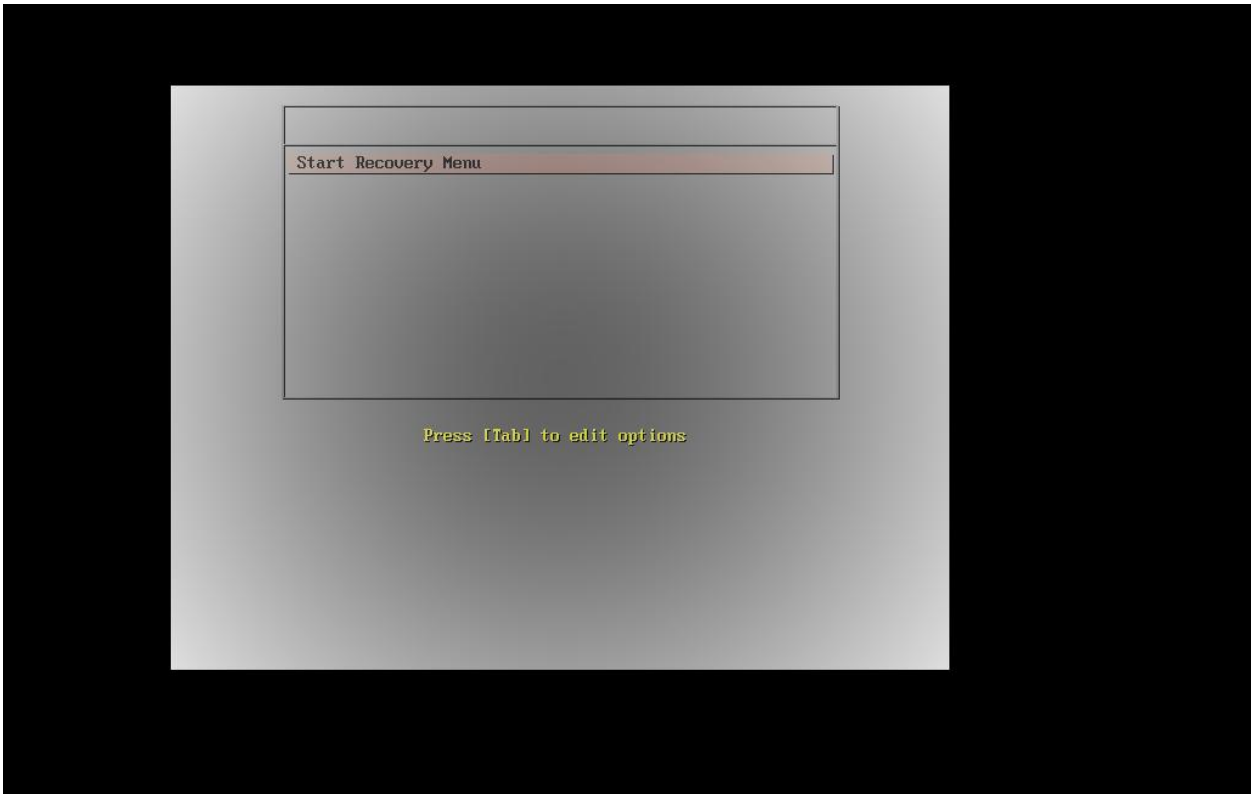
(*Install Debian 13 Minimal CLI Guide (Debian Trixie)*, 2025), (*2tUkmeDdXjM*, 2025), (*Linux Boot Menu Explained*, 2025)

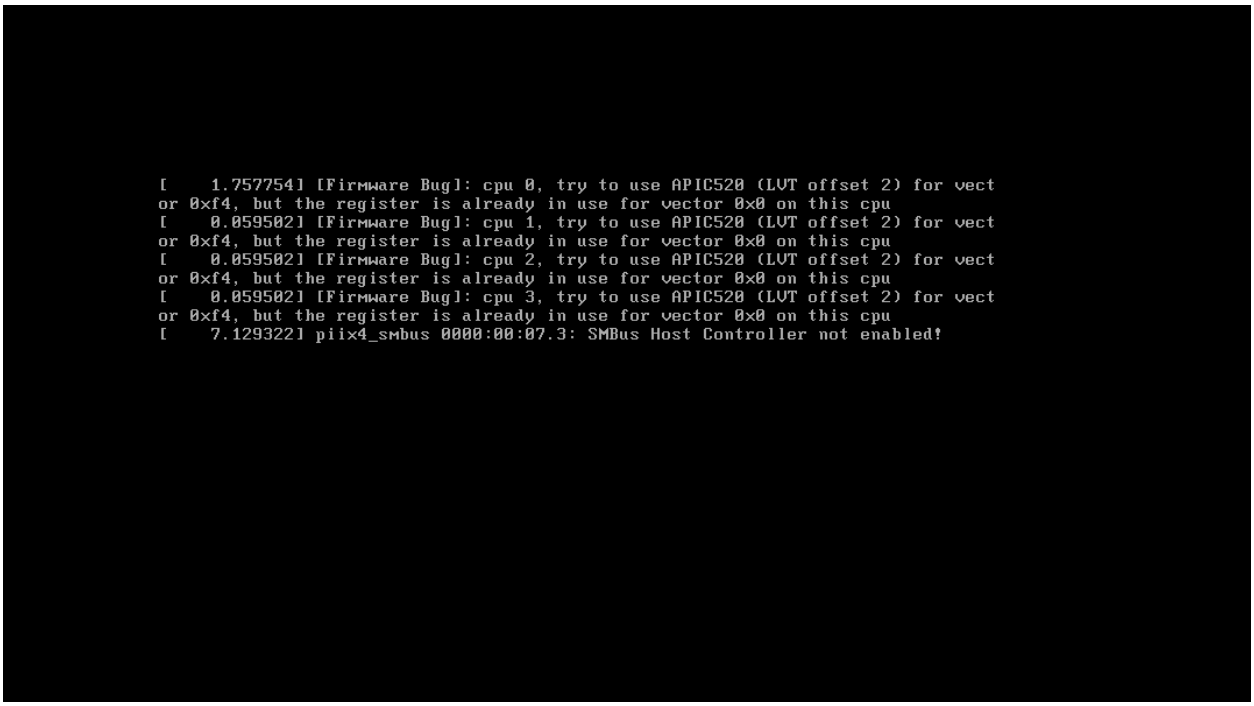
Make sure a CD/DVD is attached to the VM that points to the bootable live ISO file:



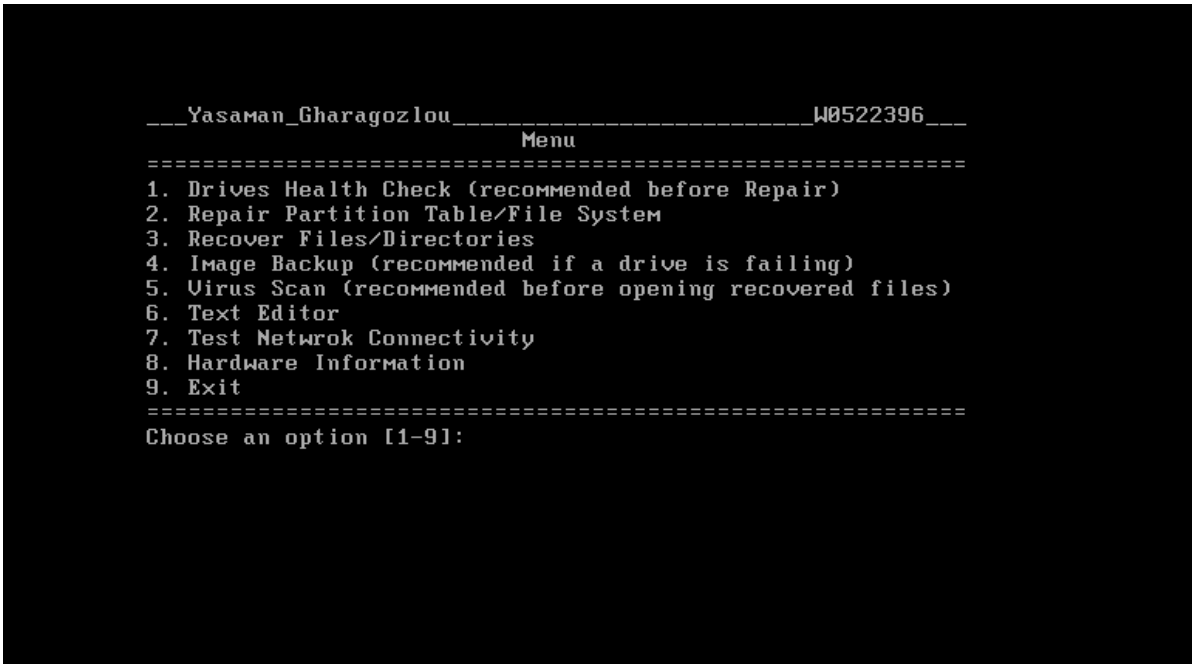
While VM is booting up, press Esc a few times to see the Boot Menu and select to boot up from CD:


Boot Menu
1. Removable Devices
2. +Hard Drive
3. CD-ROM Drive
4. Network boot from VMware VMXNET3
<Enter Setup>

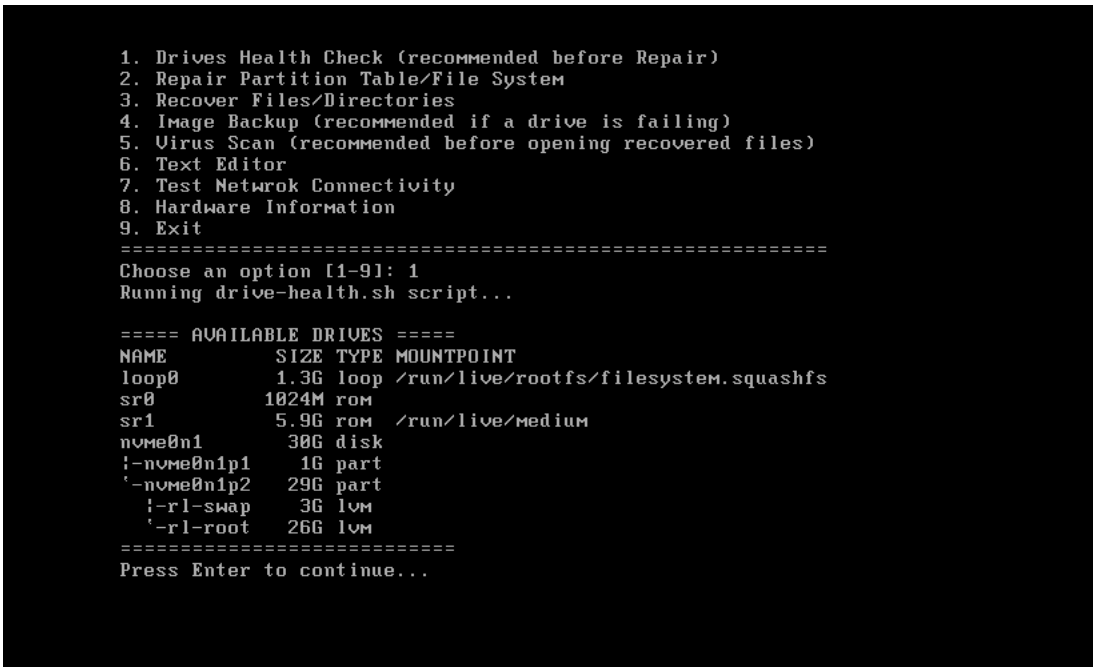

Start Recovery Menu
Press [Tab] to edit options

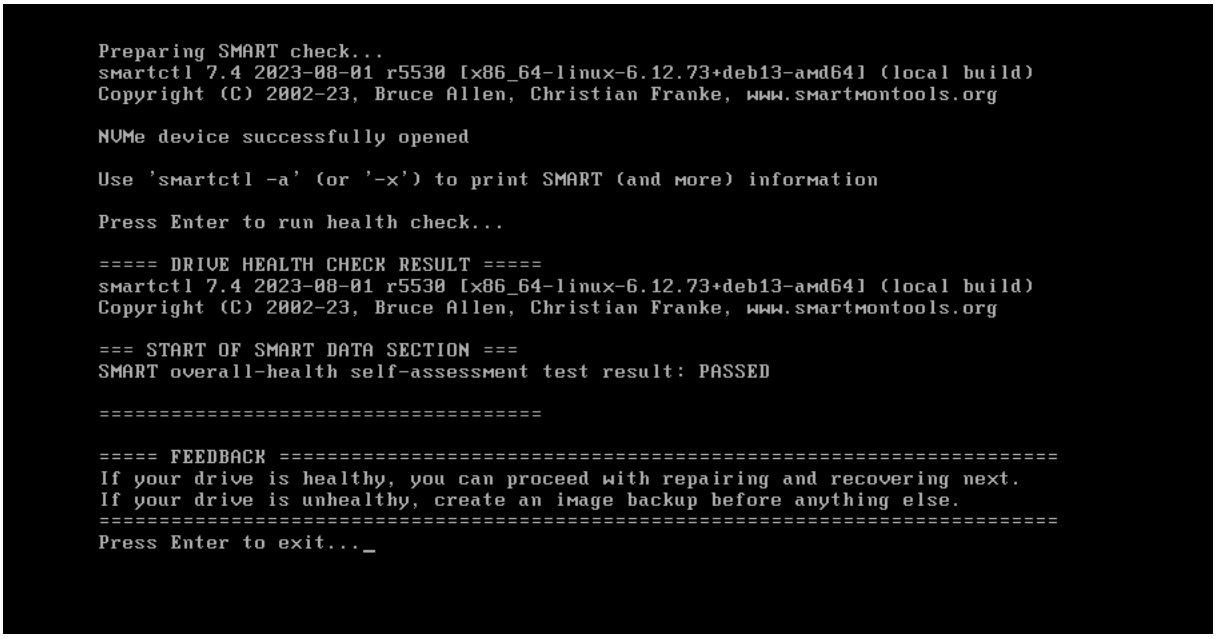


The menu immediately appears after boot:



First tool:





Second tool:

```
7. Test Netwrok Connectivity
8. Hardware Information
9. Exit
============================================================
Choose an option [1-9]: 2
Running repair.sh script...

=== DRIVES & PARTITIONS ===
NAME         SIZE TYPE MOUNTPOINT
loop0        1.3G loop /run/live/rootfs/filesystem.squashfs
sr0         1024M rom
sr1          5.9G rom  /run/live/medium
nvme0n1       30G disk
|-nvme0n1p1    1G part
`-nvme0n1p2   29G part
  |-rl-swap    3G lvm
  `-rl-root   26G lvm

Press Enter to continue...

Choose an option:
1) Repair Partition Table (uses testdisk)
2) Repair File System (uses fsck)

Enter choice (1 or 2): 1_
```

```
TestDisk 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org

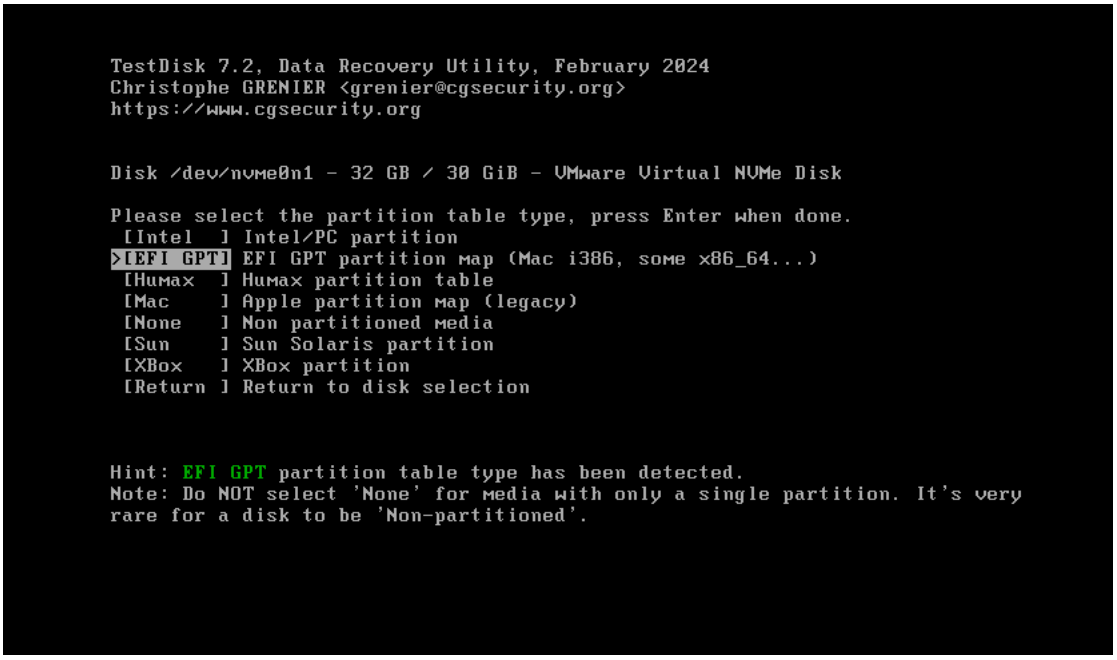
  TestDisk is free software, and
comes with ABSOLUTELY NO WARRANTY.

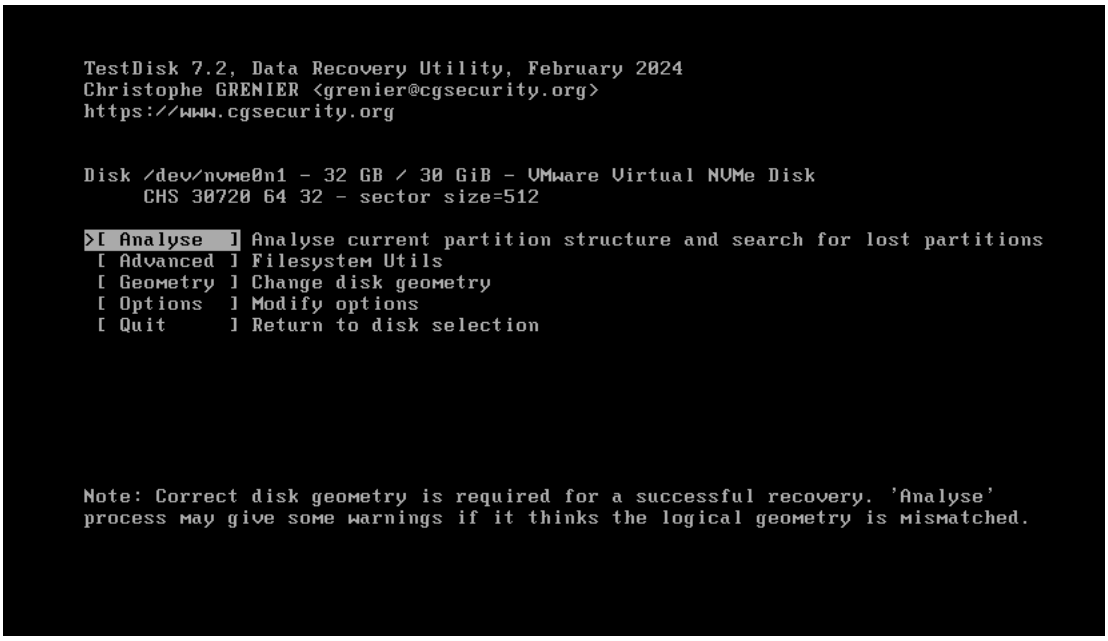
Select a media and choose 'Proceed' using arrow keys:
>Disk /dev/nvme0n1 - 32 GB / 30 GiB - VMware Virtual NVMe Disk

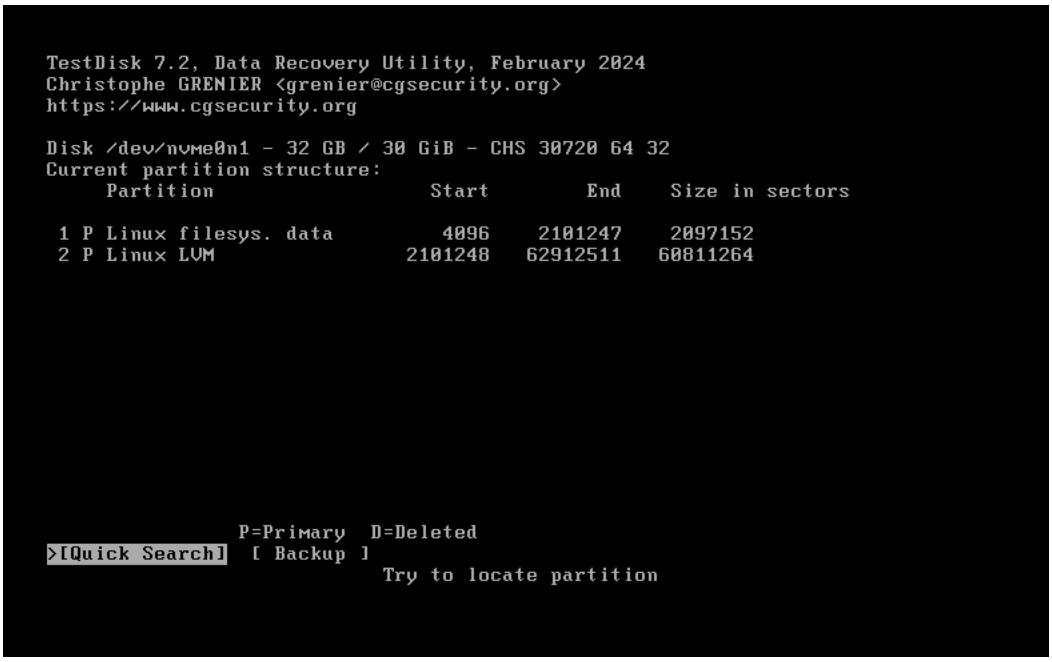


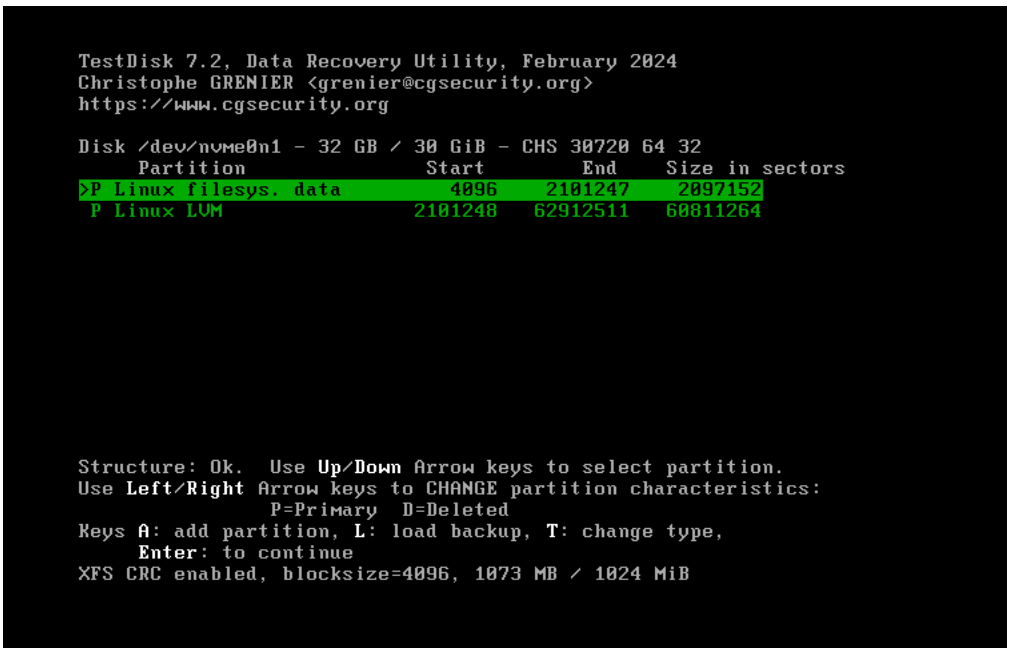

>[Proceed ]  [  Quit  ]

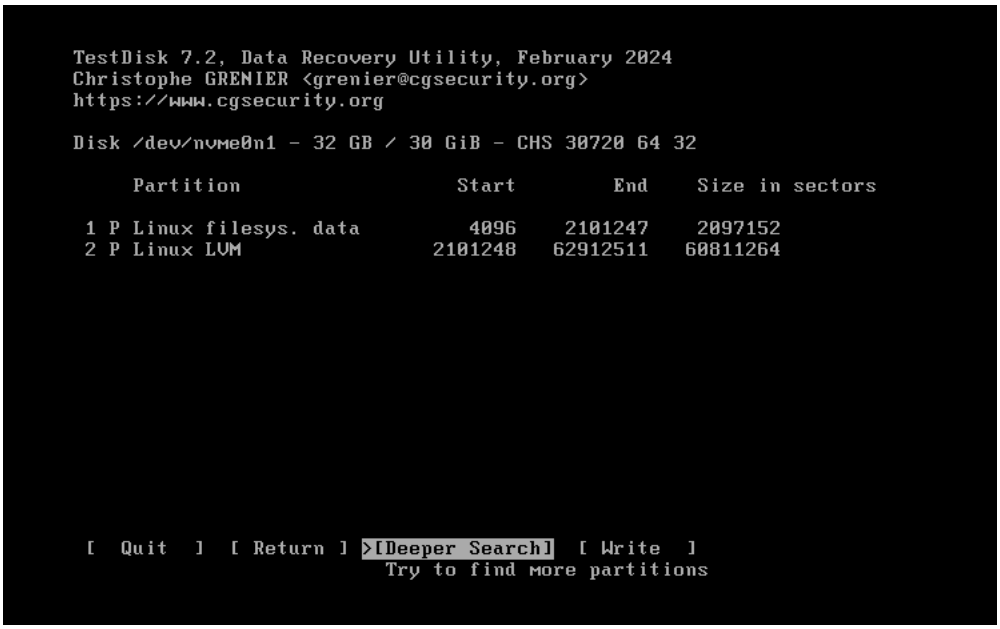
Note: Serial number VMware NVME_0000
Disk capacity must be correctly detected for a successful recovery.
If a disk listed above has an incorrect size, check HD jumper settings and BIOS
detection, and install the latest OS patches and disk drivers.
```


TestDisk 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
Disk /dev/nvme0n1 - 32 GB / 30 GiB - VMware Virtual NVMe Disk
Please select the partition table type, press Enter when done.
[Intel] Intel/PC partition
>[EFI GPT] EFI GPT partition map (Mac i386, some x86_64...)
[Humax] Humax partition table
[Mac] Apple partition map (legacy)
[None] Non partitioned media
[Sun] Sun Solaris partition
[XBox] XBox partition
[Return] Return to disk selection
Hint: EFI GPT partition table type has been detected.
Note: Do NOT select 'None' for media with only a single partition. It's very rare for a disk to be 'Non-partitioned'.

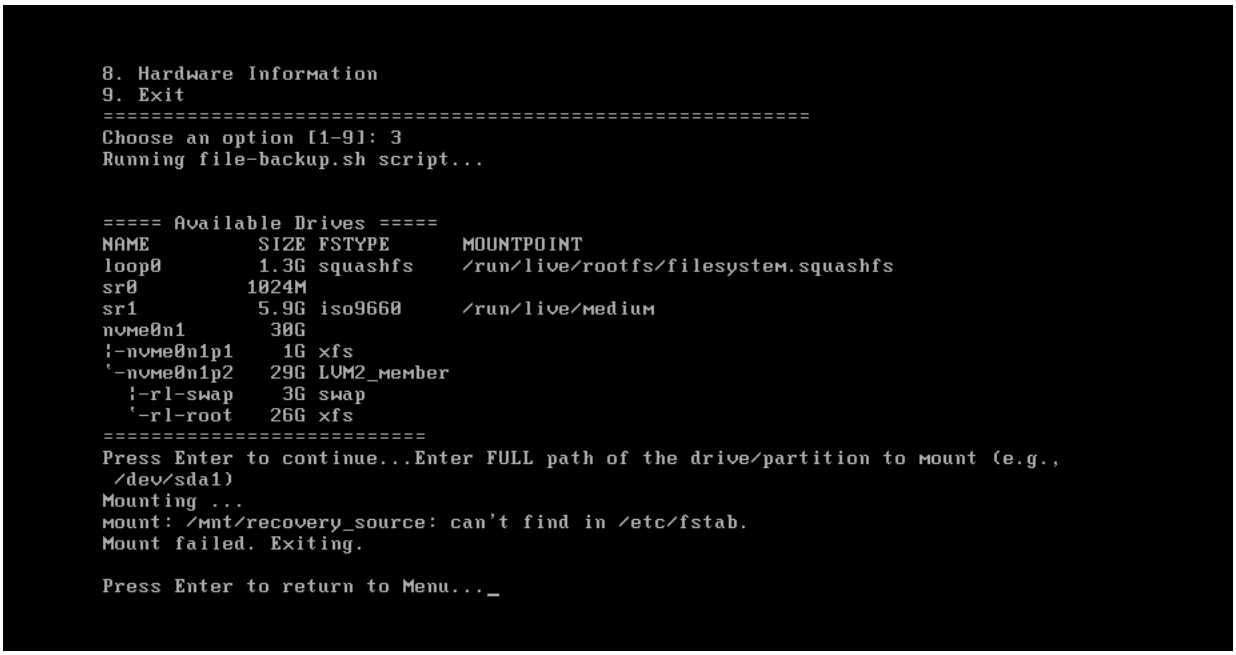

TestDisk 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
Disk /dev/nvme0n1 - 32 GB / 30 GiB - VMware Virtual NVMe Disk
CHS 30720 64 32 - sector size=512
>[Analyse] Analyse current partition structure and search for lost partitions
[Advanced] Filesystem Utils
[Geometry] Change disk geometry
[Options] Modify options
[Quit] Return to disk selection
Note: Correct disk geometry is required for a successful recovery. 'Analyse' process may give some warnings if it thinks the logical geometry is mismatched.


TestDisk 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
Disk /dev/nvme0n1 - 32 GB / 30 GiB - CHS 30720 64 32
Current partition structure:
Partition Start End Size in sectors
1 P Linux filesys. data 4096 2101247 2097152
2 P Linux LVM 2101248 62912511 60811264
P=Primary D=Deleted
>[Quick Search] [Backup]
Try to locate partition

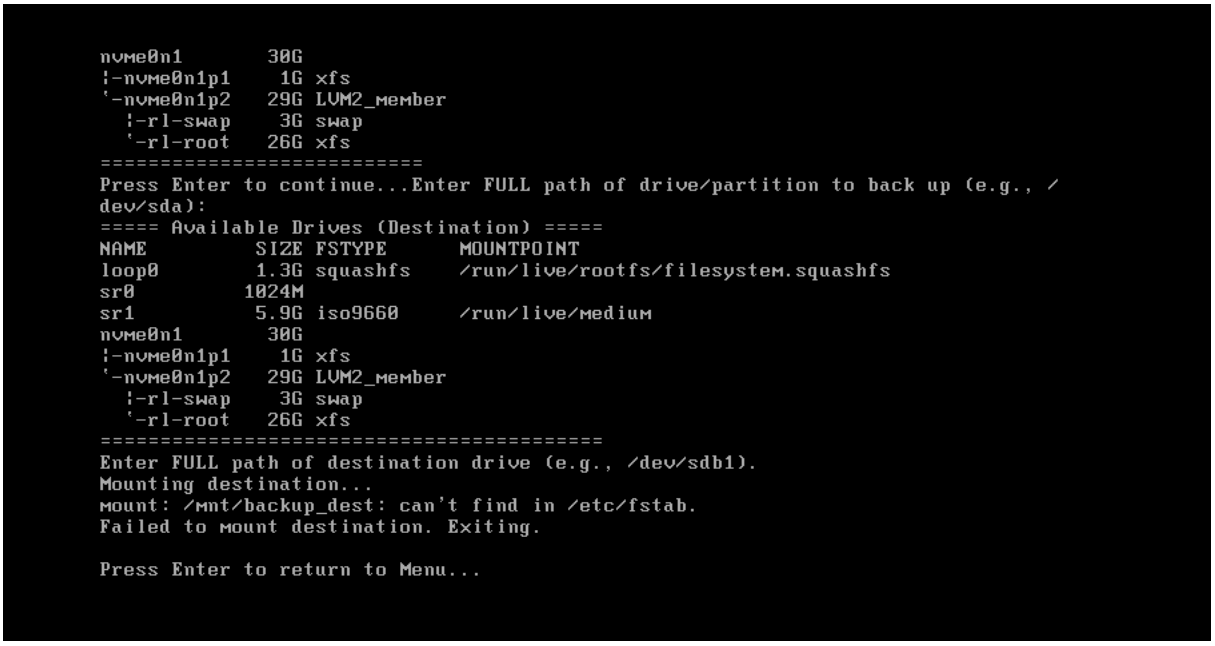

TestDisk 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
Disk /dev/nvme0n1 - 32 GB / 30 GiB - CHS 30720 64 32
Partition Start End Size in sectors
>P Linux filesys. data 4096 2101247 2097152
P Linux LVM 2101248 62912511 60811264
Structure: Ok. Use Up/Down Arrow keys to select partition.
Use Left/Right Arrow keys to CHANGE partition characteristics:
P=Primary D=Deleted
Keys A: add partition, L: load backup, T: change type,
Enter: to continue
XFS CRC enabled, blocksize=4096, 1073 MB / 1024 MiB


TestDisk 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
Disk /dev/nvme0n1 - 32 GB / 30 GiB - CHS 30720 64 32
Partition Start End Size in sectors
1 P Linux filesys. data 4096 2101247 2097152
2 P Linux LVM 2101248 62912511 60811264
[Quit] [Return] >[Deeper Search] [Write]
Try to find more partitions

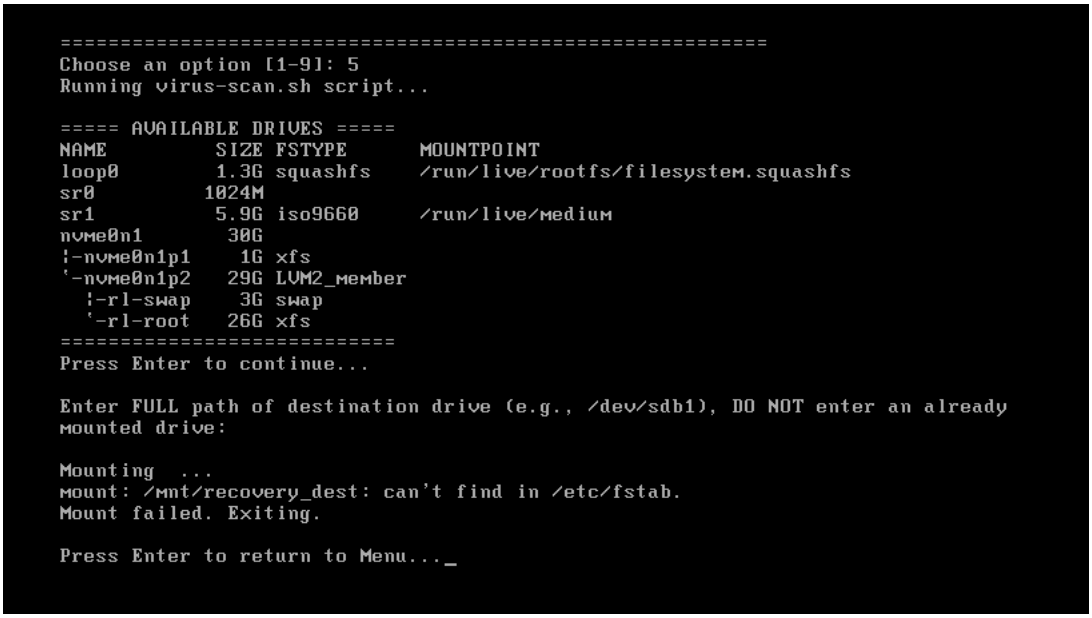
Third tool:



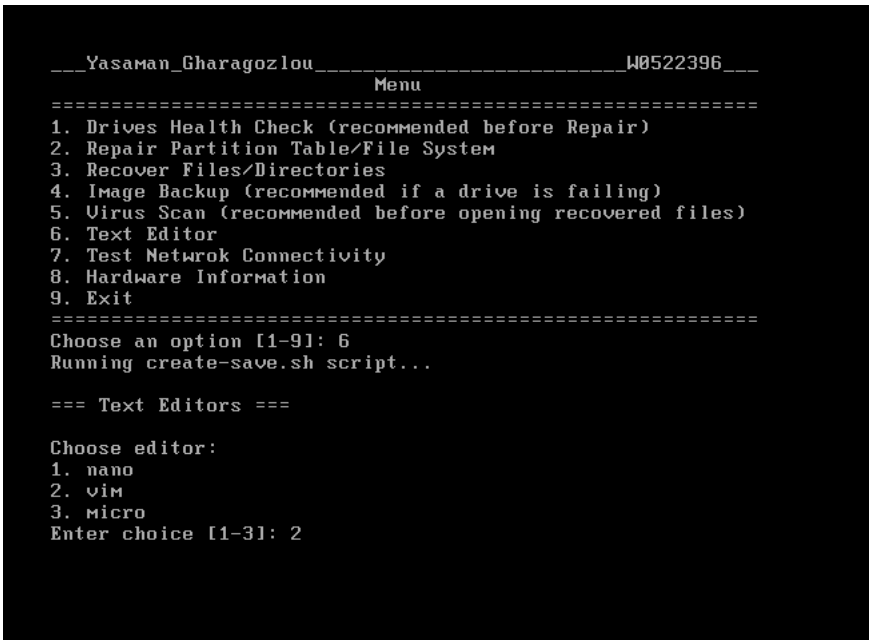
fourth tool(uses cp to copy files from one location to another, selected by the user):

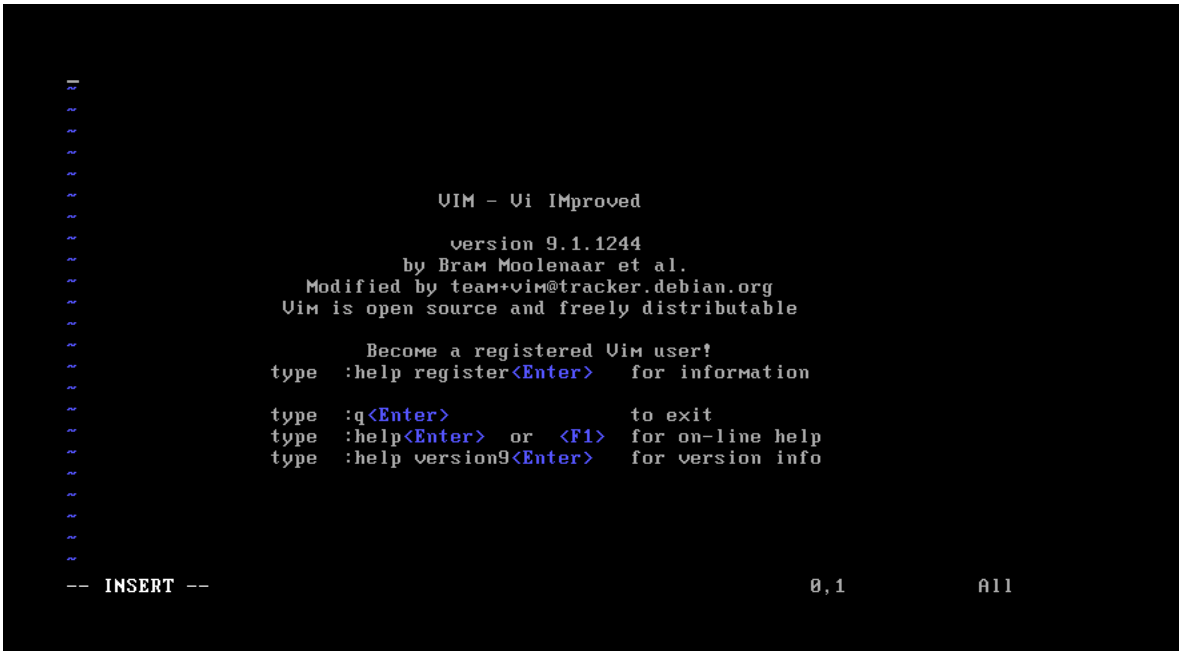


fifth tool(since the system is offline, the antivirus may throw errors while trying to reach to it's data base, but eventually will scan an entered drive and will give a feedback, I will just hit Enter for now):

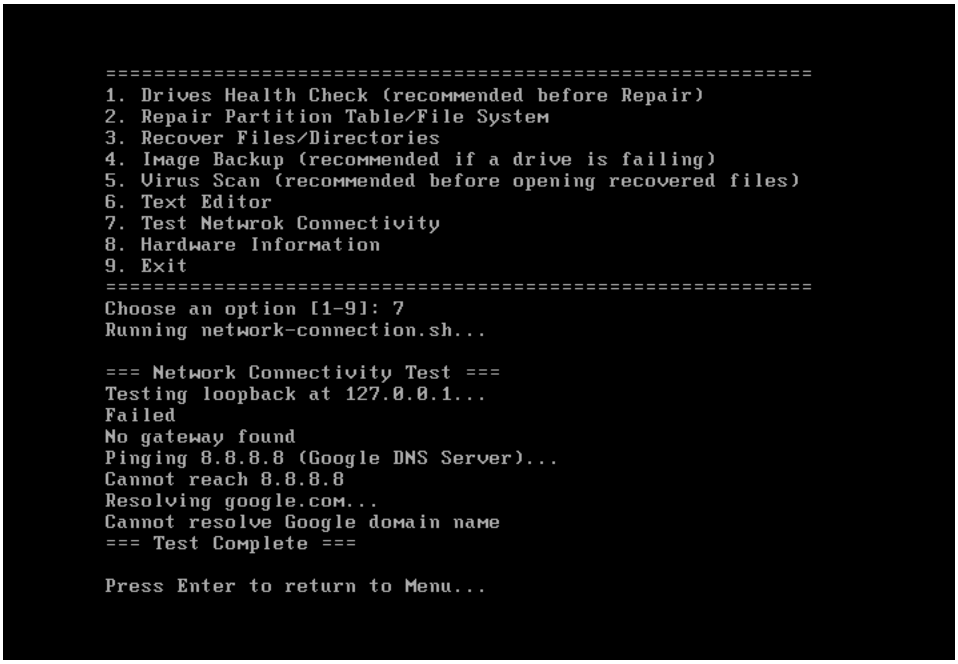


sixth tool:





seventh tool:

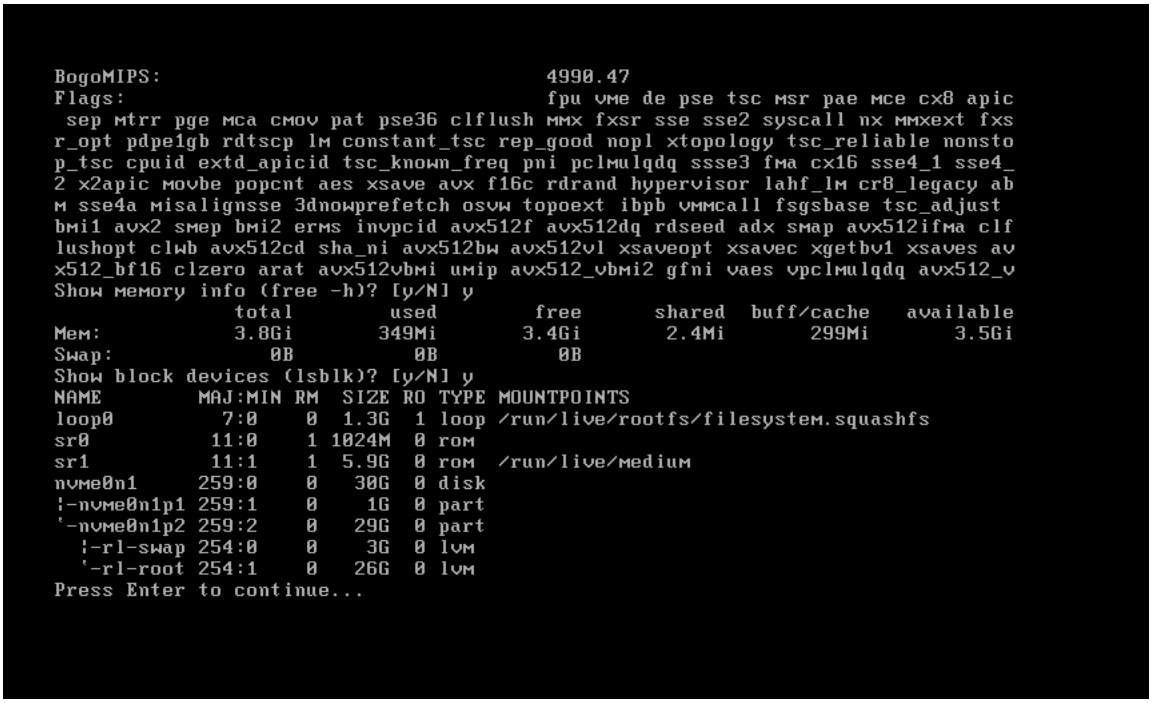


Eighth tool:

```
  Kernel: 6.12.73+deb13-amd64 arch: x86_64 bits: 64 compiler: gcc v: 14.2.0
  Console: console Distro: Debian GNU/Linux 13 (trixie)
Machine:
  Type: Vmware System: VMware product: VMware Virtual Platform v: N/A
    serial: <filter>
  Mobo: Intel model: 440BX Desktop Reference Platform serial: N/A
    BIOS: Phoenix v: 6.00 date: 03/24/2025
CPU:
  Info: 4x 1-core model: AMD Ryzen 9 8945HX with Radeon Graphics bits: 64
    type: SMP arch: Zen 4 rev: 2 cache: L1: 4x 64 KiB (256 KiB)
    L2: 4x 1024 KiB (4 MiB) L3: 4x 32 MiB (128 MiB)
  Speed (MHz): avg: 2495 min/max: N/A cores: 1: 2495 2: 2495 3: 2495 4: 2495
    bogomips: 19961
  Flags: avx avx2 lm nx pae sse sse2 sse3 sse4_1 sse4_2 sse4a ssse3
Graphics:
  Device-1: VMware SVGA II Adapter driver: N/A bus-ID: 00:0f.0
  Display: server: No display server data found. Headless machine?
    tty: 80x25
  API: EGL v: 1.5 drivers: swrast platforms: active: surfaceless,device
    inactive: gbm,wayland,x11
  API: OpenGL v: 4.5 vendor: mesa v: 25.0.7-2 note: console (EGL sourced)
    renderer: llvmpipe (LLVM 19.1.7 256 bits)
  API: Vulkan v: 1.4.309 drivers: llvmpipe surfaces: N/A devices: 1
  Info: Tools: api: eglinfo, glxinfo, vulkaninfo x11: xdriinfo, xdpyinfo,
:
```

```
H/W path         Device          Class       Description
=====================================================
                                 system      VMware Virtual Platform
/0                               bus         440BX Desktop Reference Platform
/0/0                             memory      86KiB BIOS
/0/1                             processor   AMD Ryzen 9 8945HX with Radeon Grap
hics
/0/1/0                           memory      16KiB L1 cache
/0/1/1                           memory      16KiB L1 cache
/0/1/2                           memory      1MiB L2 cache
/0/1/3                           memory      1MiB L2 cache
/0/2                             processor   AMD Ryzen 9 8945HX with Radeon Grap
hics
/0/5                             processor   AMD Ryzen 9 8945HX with Radeon Grap
hics
/0/5/95                          memory      16KiB L1 cache
/0/5/115                         memory      1MiB L2 cache
/0/6                             processor   AMD Ryzen 9 8945HX with Radeon Grap
hics
/0/6/96                          memory      16KiB L1 cache
/0/6/116                         memory      1MiB L2 cache
/0/7                             processor   CPU
/0/7/97                          memory      16KiB L1 cache
/0/7/117                         memory      1MiB L2 cache
:
```


Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 45 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 4
On-line CPU(s) list: 0-3
Vendor ID: AuthenticAMD
Model name: AMD Ryzen 9 8945HX with Radeon Graphics
CPU family: 25
Model: 97
Thread(s) per core: 1
Core(s) per socket: 1
Socket(s): 4
Stepping: 2
BogoMIPS: 4990.47
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid extd_apicid tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw topoext ibpb vmmcall fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves avx512_bf16 clzero arat avx512vbmi umip avx512_vbmi2 gfni vaes vpclmulqdq avx512_v


BogoMIPS: 4990.47
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid extd_apicid tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw topoext ibpb vmmcall fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves avx512_bf16 clzero arat avx512vbmi umip avx512_vbmi2 gfni vaes vpclmulqdq avx512_v
Show memory info (free -h)? [y/N] y
total used free shared buff/cache available
Mem: 3.8Gi 349Mi 3.4Gi 2.4Mi 299Mi 3.5Gi
Swap: 0B 0B 0B
Show block devices (lsblk)? [y/N] y
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
loop0 7:0 0 1.3G 1 loop /run/live/rootfs/filesystem.squashfs
sr0 11:0 1 1024M 0 rom
sr1 11:1 1 5.9G 0 rom /run/live/medium
nvme0n1 259:0 0 30G 0 disk
├─nvme0n1p1 259:1 0 1G 0 part
└─nvme0n1p2 259:2 0 29G 0 part
├─rl-swap 254:0 0 3G 0 lvm
└─rl-root 254:1 0 26G 0 lvm
Press Enter to continue...

Done!

Issues & solutions

Several challenges came up while working with drives, data recovery, and building a live ISO system. I made sure the virus scan database was up to date before running scans, and avoided issues caused by trying to remount drives that were already mounted. Drive health was always checked first, and I made some improvements to a drive health check script (for internal drives). In some cases, I had to use sudo to properly access and run certain repair and diagnostic tools.

During data recovery, rsync was used to handle filesystem limitations when copying files and folders. I also learned that creating a disk image with dd does not require the source drive to be mounted, which made the process more flexible. Partition-related issues were handled as well, including situations where drives had missing or corrupted partition tables.

Building the live ISO system was one of the more challenging parts. There were issues with missing packages, dependency errors, and configuration problems, so I had to troubleshoot and adjust the setup multiple times. Different methods for creating a bootable ISO were tested, each with its own drawbacks. There were also boot issues in virtual machines, such as freezing and hardware compatibility problems, which showed how important proper ISO structure and bootloader configuration are.

Comprehensive Glossary of Commands

Command	Description
dd	Creates exact disk images or clones drives at a low level (bit-by-bit), not just files.
rsync	A more advanced file copy tool that handles permissions, errors, and partial transfers useful for recovery.
fsck	Checks and repairs filesystem errors when a drive is corrupted or not mounting properly
smartctl	Shows drive health information (SMART data) to detect possible hardware issues.
clamscan	Scans files and directories for viruses.
freshclam	Updates the virus definition database.
lb config / lb build / lb clean	Used in Debian live-build to configure, build, and reset the live ISO creation process.
chroot	Allows working inside another filesystem (like the ISO root) as if it were the main system.
mount --bind	Links system directories (like /dev, /proc) into the chroot so tools function correctly.
unsquashfs	Extracts the compressed root filesystem (filesystem.squashfs) from the ISO so it can be modified.
mksquashfs	Repackages the modified root filesystem back into a compressed .squashfs file for the ISO.
xorriso	Used to create or modify ISO images. It helps package all the files (including the bootloader) into a bootable ISO file.
testdisk	Used for recovering lost partitions and fixing partition tables. It was helpful when dealing with drives that had missing or corrupted partition information.
lscpu	Displays CPU information such as cores, architecture, and model.
inxi	Gives a quick, readable summary of system hardware and status.
free	Shows memory (RAM) usage.
ip addr	Shows the IP addresses assigned to your network interfaces.
ip a	Short version of ip addr, used to quickly view IP information
ip addr add	Assigns a temporary IP address to a network interface.

ip route	Displays or sets routing information, such as the default gateway.
nmcli	Lists all available network connections.
nmcli connection show	Shows detailed information about all running processes on the system.
nmcli connection up	Activates (connects) a network connection.
ping	Tests if another device or website is reachable over the network.
traceroute	Shows the path that data takes to reach a destination.
dig	Performs detailed DNS lookups.
nslookup	Queries DNS to find the IP address of a domain (simpler than dig).

adduser	Creates a new user and automatically sets up a home directory.
useradd	Creates a new user; requires the -m option to also create a home directory.
usermod	Modifies a user's properties (e.g., name, groups). Use -a with -G to append groups.
passwd	Sets or changes the password for a user.
groupadd	Creates a new group.
su	Switches from the current user to another user (requires that user's password).
sudo	Runs a command with root (administrator) privileges.
id	Displays user information such as UID, GID, and group memberships.
tcpdump	Captures and displays network packets; typically requires sudo to run.

sleep	Pauses execution for a specified amount of time (e.g., sleep 5 pauses for 5 seconds).

tee	Reads input and writes it to both the terminal and a file at the same time.
read	Accepts user input and stores it in a variable (commonly used in shell scripts).
clear	Clears the terminal screen.
df -h --total	Displays disk usage in human-readable format (KB, MB, GB) and shows a total summary.
ps -ef	Shows detailed information about all running processes on the system.
ps -u $USER	Displays processes that belong to the current logged-in user.
chmod +x	Adds execute permission to a file, making it runnable as a script or program.
touch	Creates a new file.
cat	Displays file content. cat > filename creates and writes to a file; cat >> filename appends.
less	Views a file one screen at a time; allows searching.
head	Shows the first lines of a file. Example: head -n 18 shows first 18 lines.
sort	Sorts lines in a file alphabetically.
grep	Searches for text patterns. -i = ignore case, -c = count matches, -E = extended regex.
sed	Stream editor to search and replace text in files.
wc	Counts lines, words, or characters. wc -c counts characters.
stat	Displays metadata about a file (size, permissions, timestamps).
chmod	Changes file/directory permissions. chmod 1777 sets sticky bit; chmod +t adds sticky bit.
umask	Sets default permissions for new files and directories.
vi	Modal text editor for creating and editing files.
help	Shows brief help for shell built-in commands
man	Displays the full manual page for a command
whoami	Displays the username of the currently logged-in user
who	Shows all users currently logged into the system
pwd	Prints the current working directory
ls / ls -l / ls c*	Lists directory contents, with details or pattern matching
cd / cd .. / cd ~	Changes directories (parent directory or home directory)
tree -d -L 2	Displays directories only, up to two levels deep
echo	Outputs text to the terminal or a file
which	Shows the full path of the executable used for a command
head / tail	Displays the beginning or end of a file

apropos	Searches manual page descriptions for related commands
chmod	Changes file or directory permissions
sudo	Executes commands with elevated (administrator) privileges
shutdown	Schedules or performs a system shutdown
ln -s / ln	Creates a link for a given file
du	Shows information about disk usage of specific given files
df	Shows information about disk usage of given filesystem
sudo apt-get update	To update the lists of the packages that need upgrading as well as the newer packages in the repository (question), 2012)
sudo apt-get install open-vm-tools-desktop	To both install and upgrade open-vm-tools (documentation) B. I., 2025)
systemctl status open-vm-tools.service	To make sure VMWare tools are implemented using open-vm-tools as an open source suit/service. (documentation) B. I., 2025)
sudo hostnamectl set-hostname CLYG01 (liam, 2023)	To change my hostname (liam, 2023)

References

Debian Install Notes. (2020). *Live-build*. https://debian-install-notes.pages.dev/netinstall/live-build

Rocky Linux Documentation. (n.d.). *Xorriso*. https://docs.rockylinux.org/10/gemstones/xorriso/

Linux Kernel Documentation. (n.d.). *Squashfs filesystem*. https://docs.kernel.org/filesystems/squashfs.html

Red Hat, Inc. (n.d.). *Boot menu customization*. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/anaconda_customization_guide/sect-boot-menu-customization

Install Debian 13 Minimal CLI Guide (Debian Trixie). (2025, August 3). YouTube. https://www.youtube.com/watch?v=FlSbx7jg1WQ&list=PLfj-JUKi_Yp8vBPJTcjxM9Tt35lCrcyS4

2tUkmeDdXjM [Video]. (2025). YouTube. https://www.youtube.com/watch?v=2tUkmeDdXjM&list=PLfj-JUKi_Yp8vBPJTcjxM9Tt35lCrcyS4&index=2

Linux Boot Menu Explained [Video]. (2025, July 15). YouTube. https://www.youtube.com/watch?v=RmagSaz84tE&list=PLfj-JUKi_Yp8vBPJTcjxM9Tt35lCrcyS4&index=3

ICOP Technology Inc. (2024, December 27). *How to install and configure Debian NTFS & exFAT support*. https://www.icop.com.tw/news/3074